FUSION Boot Camp 2026 — סדרת מחברות הסבר¶

29th International Conference on Information Fusion
Trondheim, Norway · June 22–26, 2026
מתאם: Dale Blair · מרצים: Blasch, Blair, Govaers, Meyer, Bar-Shalom, Varshney

סדרת מחברות זו מתעדת ומדגימה בקוד את הפיתוחים (developments) שתוארו בקובץ `FUSION Boot Camp 2026.pdf`. הקובץ עצמו מכיל שני מערכי שקופיות מלאים — המבוא של Erik Blasch ושיעור האמידה (Estimation) של W. Dale Blair — וכן את לוח הזמנים של שבע ההרצאות של ה-Boot Camp.

לוח הזמנים של ה-Boot Camp (22 ביוני 2026)¶

# נושא	מרצה
1 Introduction to Information Fusion	Erik Blasch
2 Estimation in Information Fusion	Dale Blair
3 Data Association	Florian Meyer
4 Distributed Target Tracking	Felix Govaers
5 Fusion and Association with Attributes/Classification	Yaakov Bar-Shalom
6 Distributed Inference and Information Fusion	Pramod Varshney
7 High-level Fusion	Erik Blasch

מבנה הסדרה¶

מחברת	תוכן	מבוסס על
`00_overview.ipynb`	סקירה כללית, לוח זמנים, אינדקס	—
`01_introduction_to_information_fusion.ipynb`	מושגי יסוד, היסטוריה, מודל JDL/DFIG (רמות 0–5), משוואת Fusion-ה, יתרונות/חסרונות, היררכיית ההיסק	הרצאת Blasch
`02_parameter_estimation.ipynb`	ML, MAP, MMSE, LSE, WLSE, חסם Cramér–Rao, אמידת הטיה (bias)	הרצאת Blair (חלק א')
`03_state_estimation_kalman_filter.ipynb`	גזירת מסנן קלמן, predictor–corrector, רעש תהליך DWNA/CWNA, מודל NCV	הרצאת Blair (חלק ב')
`04_target_tracking_maneuvering.ipynb`	NCV מול NCA, בחירת שונות רעש התהליך, IMM, EKF, ריבוי חיישנים	הרצאת Blair (חלק ג')
`05_data_association.ipynb`	Gating, GNN, PDA/JPDA, MHT	הרצאת Meyer*
`06_distributed_target_tracking.ipynb`		

מחברת	תוכן	מבוסס על
	מסנן מידע, track-to-track, Covariance Intersection	הרצאת *Govaers
07_attributes_classification.ipynb	סיווג בייסיאני רצוף, מטריצת בלבול, Dempster–Shafer	הרצאת Bar-Shalom*
08_distributed_inference.ipynb	ROC, כללי k-out-of-N, Chair–Varshney	הרצאת *Varshney
09_high_level_fusion.ipynb	רמות L2–L5, הערכת מצב/איום, MOP/MOE, hard+soft	הרצאת *Blasch (7)

הערה (*): ההרצאות 3–7 מופיעות בלוח הזמנים אך השקופיות שלהן **אינן** כלולות ב-PDF. מחברות 05–09 הן **מחברות מלוות** המבוססות על הספרות הקנונית של התחום (Bar-Shalom, Blackman & Popoli, Varshney, Blasch) ועל ההקשר שמספק Blasch במבוא — ולא על שקופיות מקוריות. מחברות 01–04 מבוססות ישירות על שני המערכים שב-PDF.

דרישות הרצה¶

```
pip install numpy matplotlib
```

כל מחברת עצמאית וניתנת להרצה מתחילתה לסופה (Run All).

```
# בדיקת סביבה
import sys
import numpy as np
import matplotlib
print('Python', sys.version.split()[0])
print('numpy', np.__version__)
print('matplotlib', matplotlib.__version__)
```

```
Python 3.11.15
numpy 2.4.6
matplotlib 3.11.0
```

01 · מבוא ל-Information Fusion¶

מבוסס על: Erik Blasch, FUSION Boot Camp 2026 — Introduction to Data Fusion.

Take-aways של ההרצאה¶

1. **מהו Data Fusion?** — היתוך מקטין אי-ודאות; הכמת אי-הוודאות היא השונות P (covariance).
2. **שיטות:** רישום נתונים (registration) = קורלציה; היתוך נתונים (fusion) = שיוך נתונים (association).
3. **מה fusion יכול ולא יכול לעשות:** מרחיב כיסוי במרחב/זמן ומקטין עומס נתונים; אינו 'מספר קסם', אינו מתקן נתונים גרועים, ואינו מחליף הסקה אנושית.
4. **ארכיטקטורות:** מודל JDL/DFIG, רמות 0–5 (טקסונומיה); חישה רב-תחומית (חלל, אוויר, סייבר) המשלבת hard (תמונות) ו-soft (טקסט).

א. היסטוריה קצרה של רעיון ה-Fusion¶

Blasch ממקם את היתוך המידע ברצף היסטורי ארוך של 'איחוד חושים' והסקה:

תקופה	תרומה
יוונים	אפלטון — טענות/אמונות/ראיות; אריסטו — הנשמה כמאחדת חושים
מטריאליזם	דקארט — גוף ונפש; אינטראקציה סיבתית (חומר ותנועה)
1500–1700	קפלר, גלילאו, ניוטון — תנועה ואינטראקציה דינמית
1700	Bayes — אמונות (probability)
1800	ניסויים — תגובה לגירוי
1900	Galton, Pearson — קורלציה (שיוך); William James — איחוד חושים (FUSE)
1960	**מסנן קלמן (Kalman Filter)**
1990	סדרת הספרים הראשונה על Sensor & Data Fusion

Fusion כמדע: שלושה שערים¶

- **אונטולוגיה (Ontology)** — מהו 'העולם' ומבניו (הצהרה על מה שקיים; למשל החישה).
- **אפיסטמולוגיה (Epistemology)** — כללי המשחק; כיצד יודעים משהו (כללי ה-fusion).
- **טקסונומיה (Taxonomy)** — סיווג/סמנטיקה של המידע.

ב. סוגי היסק (Inference) ב-Fusion ברמה גבוהה¶

	Inductive	Deductive	Abductive
פעולה	מהתבוננות להבנה כללית	גזירת תוצאה מתוך חוק	היסק ההסבר הסביר ביותר לתצפית
כיוון	מהפרטי לכללי	מהכללי לפרטי	שילוב יחסים
שיטות	Bayes Nets If-then, Hypothesis tests Probability, Bayesian, entropy		

(E. Blasch, P. Valin, E. Bossé, "Measures of Effectiveness for High-Level Fusion," Fusion 2010.)

ג. משוואת ה-Fusion המרכזית (Key Fusion Equation)¶

הטענה המרכזית: **fusion מקטין אי-ודאות**. כאשר ממזגים שתי מדידות בלתי-תלויות של אותו ערך, עם שונויות P_1, P_2 (או σ_1^2, σ_2^2), מתקבל אומדן ממוזג ש**השונות שלו קטנה מכל אחת מהן**:

$$\frac{1}{P_F} = \frac{1}{P_1} + \frac{1}{P_2} \quad \Rightarrow \quad P_F = \frac{P_1 P_2}{P_1 + P_2}$$

(adapted from Maybeck, 1978). זוהי בדיוק נוסחת מיזוג שני גאוסיאנים — שקולה לעדכון מדידה במסנן קלמן.

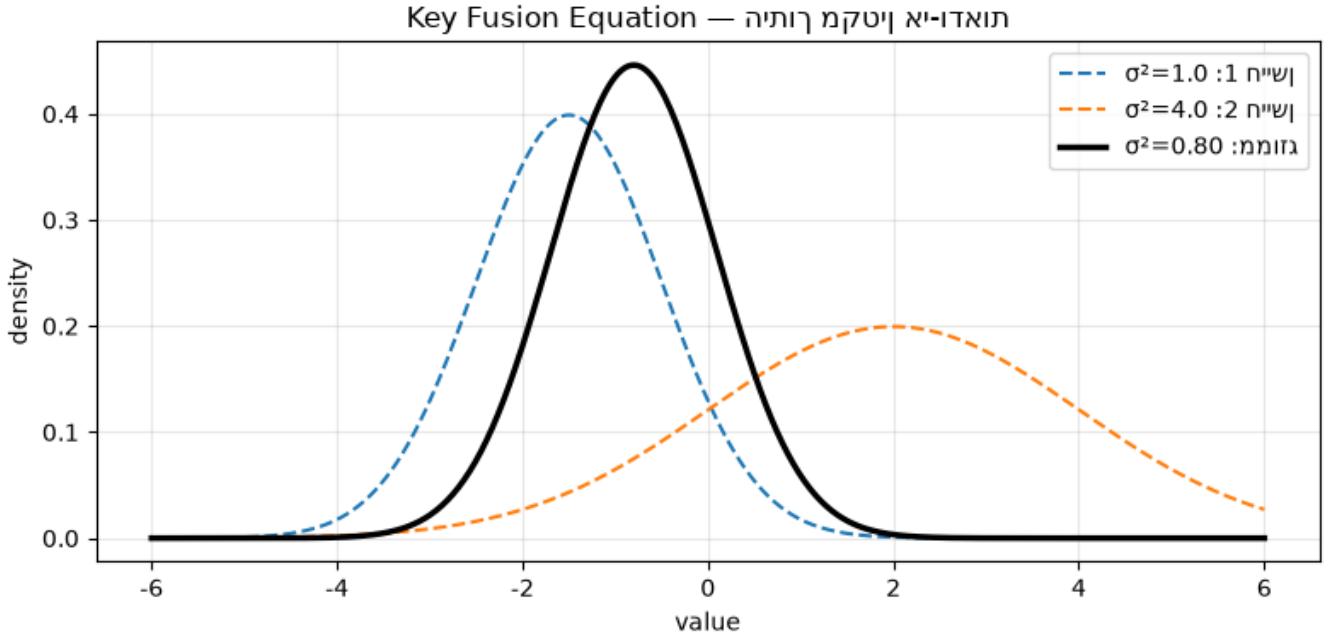
דוגמת השקופית: עם $\sigma_1^2 = 1$ ו-$\sigma_2^2 = 4$ מתקבל $P_F = \frac{1 \cdot 4}{1+4} = 0.8$, כלומר $\sigma_F = \sqrt{0.8} \approx 0.90$ — קטן מ-$\sigma_1 = 1$.

```
import numpy as np
import matplotlib.pyplot as plt

def fuse(x1, P1, x2, P2):
    """(סקלר או מטריצה) מיזוג שני אומדנים גאוסיאניים בלתי-תלויים.
    מחזיר את האומדן הממוזג ואת השונות הממוזגת."""
    P1, P2 = np.asarray(P1, float), np.asarray(P2, float)
    if P1.ndim == 0:  # סקלר
        PF = (P1 * P2) / (P1 + P2)
        xF = PF * (x1 / P1 + x2 / P2)
        return xF, PF
    PF = np.linalg.inv(np.linalg.inv(P1) + np.linalg.inv(P2))  # מיזוג ע"י מידע (information
form)
    xF = PF @ (np.linalg.inv(P1) @ x1 + np.linalg.inv(P2) @ x2)
    return xF, PF

# דוגמת השקופית
xF, PF = fuse(0.0, 1.0, 0.0, 4.0)
print(f'P_F = {PF:.3f},  sigma_F = {np.sqrt(PF):.3f}  (קטן מ-sigma_1=1 ומ-sigma_2=2)')
```

```
P_F = 0.800,  sigma_F = 0.894  (קטן מ-sigma_1=1 ומ-sigma_2=2)
```

```
# וידואליזציה: שני גאוסיאנים והגאוסיאן הממוזג
from math import sqrt, pi
```

```
x = np.linspace(-6, 6, 400)
def gauss(x, mu, var):
    return np.exp(-(x-mu)**2/(2*var))/sqrt(2*pi*var)

x1, P1, x2, P2 = -1.5, 1.0, 2.0, 4.0
xf, Pf = fuse(x1, P1, x2, P2)
plt.figure(figsize=(8,4))
plt.plot(x, gauss(x, x1, P1), '--', label=f'1 חיישן: σ²={P1}')
plt.plot(x, gauss(x, x2, P2), '--', label=f'2 חיישן: σ²={P2}')
plt.plot(x, gauss(x, xf, Pf), 'k', lw=2.5, label=f'ממוזג: σ²={Pf:.2f}')
plt.title('Key Fusion Equation — היתוך מקטין אי-ודאות')
plt.xlabel('value'); plt.ylabel('density'); plt.legend(); plt.grid(alpha=.3)
plt.tight_layout(); plt.show()
print('שונות קטנה יותר <- שיא הצפיפות הממוזגת גבוה וצר יותר :לב שים.')
```



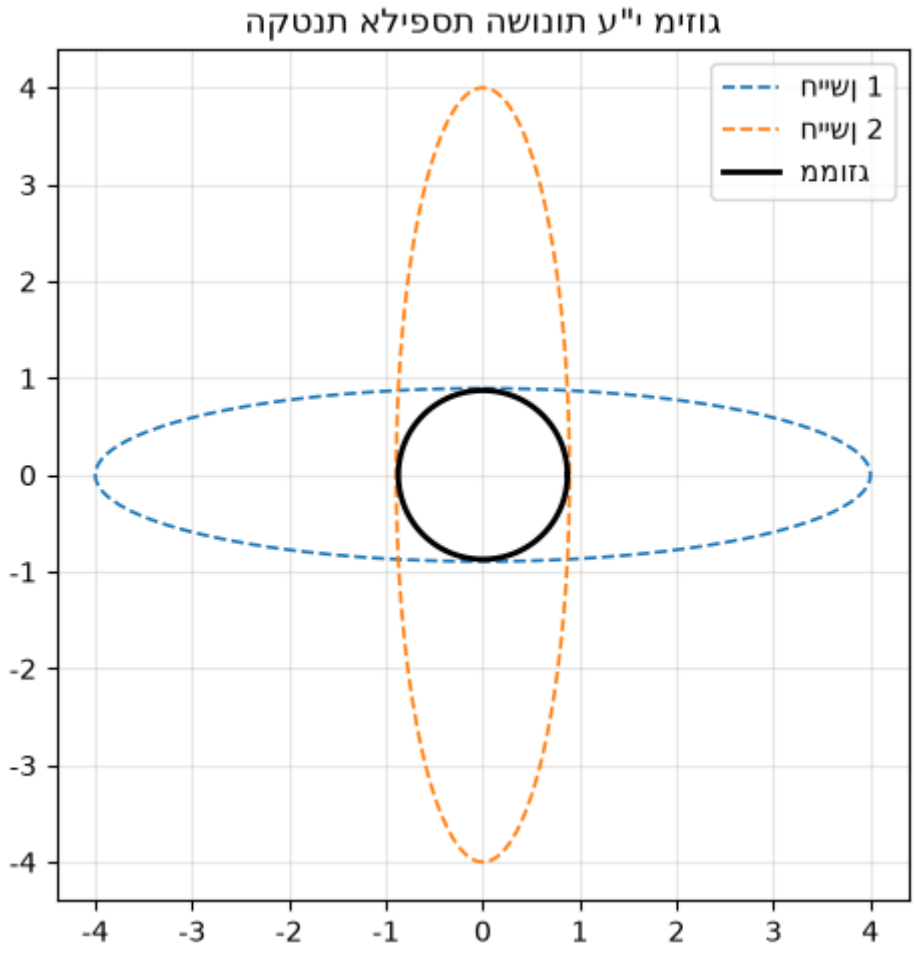
שים לב: שיא הצפיפות הממוזגת גבוה וצר יותר <- שונות קטנה יותר.

הקטנת אליפסת השונות (Covariance Ellipse Reduction) — מקרה דו-ממדי¶

כאשר שני חיישנים מודדים מיקום במישור עם אליפסות שגיאה שונות (למשל רדאר מדויק בטווח ולא בזווית, וחיישן אופטי להפך), המיזוג מצמצם את שטח האליפסה — בדיוק העיקרון שמאחורי טריאנגולציה.

```
def cov_ellipse(P, mean=(0,0), n_std=2.0, **kw):
    vals, vecs = np.linalg.eigh(P)
    t = np.linspace(0, 2*np.pi, 200)
    circle = np.array([np.cos(t), np.sin(t)])
    ell = vecs @ np.diag(n_std*np.sqrt(vals)) @ circle
    return ell[0]+mean[0], ell[1]+mean[1]

P1 = np.array([[4.0, 0.0],[0.0, 0.2]])   # מדויק לאורך y, גס לאורך x
P2 = np.array([[0.2, 0.0],[0.0, 4.0]])   # להפך
xF, PF = fuse(np.zeros(2), P1, np.zeros(2), P2)

plt.figure(figsize=(5,5))
for P, lab, st in [(P1,'1 חיישן','C0--'),(P2,'2 חיישן','C1--'),(PF,'ממוזג','k-')]:
    ex, ey = cov_ellipse(P)
```

```
    plt.plot(ex, ey, st, lw=2 if lab=='ממוזג' else 1.2, label=lab)
plt.gca().set_aspect('equal'); plt.legend(); plt.grid(alpha=.3)
plt.title('הקטנת אליפסת השונות ע"י מיזוג'); plt.tight_layout(); plt.show()
print('det(P1)=%.3f det(P2)=%.3f det(P_F)=%.3f  (שטח האליפסה הממוזגת קטן בהרבה)'
      % (np.linalg.det(P1), np.linalg.det(P2), np.linalg.det(PF)))
```




```
det(P1)=0.800 det(P2)=0.800 det(P_F)=0.036  (שטח האליפסה הממוזגת קטן בהרבה)
```

ד. מודל ה-JDL/DFIG — רמות ה-Fusion 0–5¶

הטקסונומיה הסטנדרטית לתיאור פונקציות ה-fusion (JDL במקור, הורחב ל-DFIG ב-2006):

	רמה שם	תיאור
L0	Sub-object / Signal	רישום ועיבוד אות, חילוץ מאפיינים
L1	Object assessment	גילוי, זיהוי (classification), ומעקב (tracking) של עצמים
L2	Situation assessment	אמידת יחסים בין ישויות; תמונת מצב
L3	Threat / Impact assessment	אמידת איום, כוונה (intent), והשלכות עתידיות
L4	Process refinement	ניהול חיישנים ובקרה (sensor management)
L5	User refinement	האדם בלולאה — הצגה, אינטראקציה, החלטה

.DFIG (Data Fusion Information Group), Situation Assessment", J. of Adv. in Info. Fusion, Dec") .2006)

מה אפשר היום (Can Do): רישום תמונה (L0), גילוי/זיהוי ומעקב מטרות לא-מתמרנות בסביבה דלילה (L1), הסקה אוטומטית מוגבלת ב-L2/L3, אופטימיזציה לחיישנים תואמים ב-L4, ואספקת אומדנים לאדם ב-L5.
מה עדיין קשה (Can NOT Do): מעקב מדויק אחר מטרות מתמרנות בצפיפות גבוהה, התקרבות ליכולת ההסקה האנושית, מידול מדויק של כוונה, ואופטימיזציה של חיישנים לא-תואמים מבוזרים.

```
# היררכיית ההיסק (Inference Hierarchy) — מרמת אות לרמת ידע
levels = [
    ('L0 אות',        'Signal Processing',     'גילוי, מאפיינים'),
    ('L1 עצם',        'Estimation',            'Kalman/Bayesian, MAP'),
    ('L2 מצב',        'Decision-level',        'Bayesian nets, fuzzy, NN'),
    ('L3 איום',       'Knowledge-based',       'מערכות מומחה, CBR'),
    ('L4/L5 ניהול',   'Control / User',        'sensor mgmt, human-in-loop'),
]
print(f"{'רמה':<12}{'טכניקות':<22}{'דוגמאות'}")
print('-'*60)
for lv, tech, ex in levels:
    print(f'{lv:<12}{tech:<22}{ex}')
```

```
דוגמאות                טכניקות         רמה
------------------------------------------------------------
L0 אות       Signal Processing     גילוי, מאפיינים
L1 עצם       Estimation            Kalman/Bayesian, MAP
L2 מצב       Decision-level        Bayesian nets, fuzzy, NN
L3 איום      Knowledge-based       מערכות מומחה, CBR
L4/L5 ניהול  Control / User        sensor mgmt, human-in-loop
```

ה. יתרונות וחסרונות¶

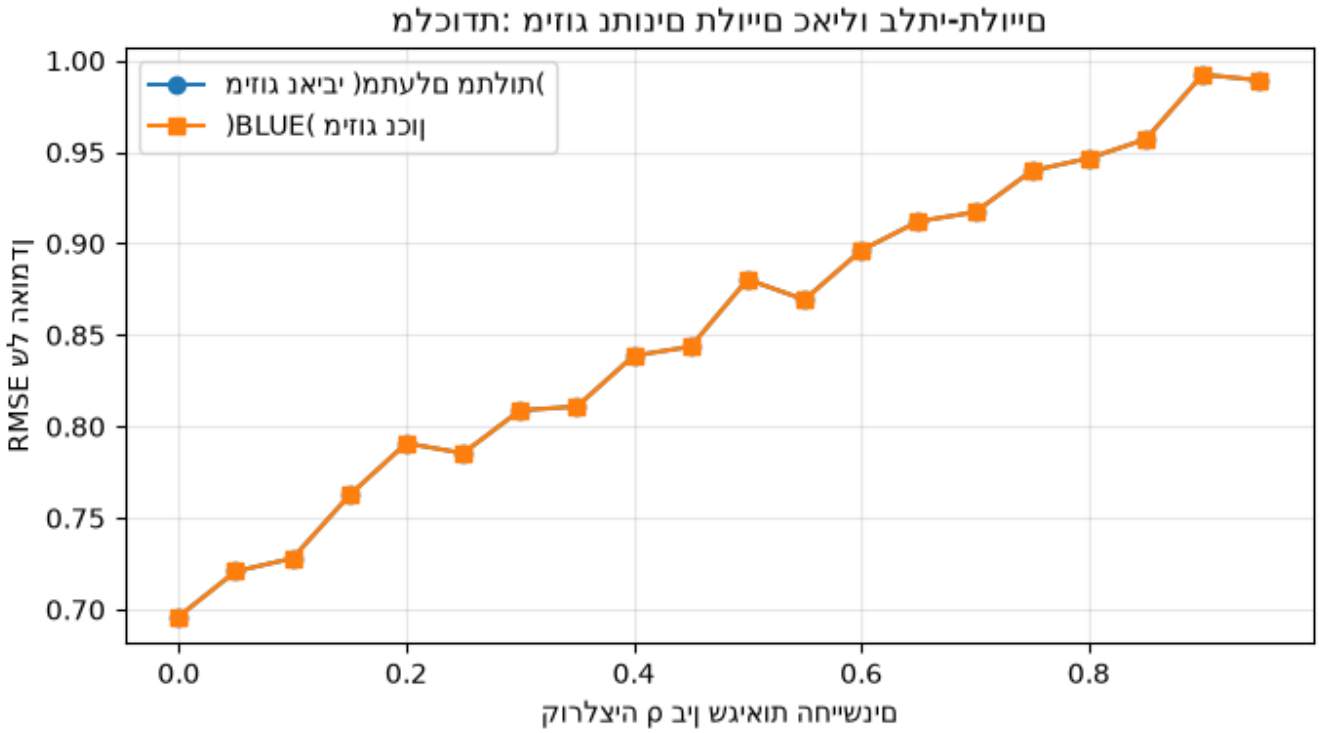
יתרונות (Waltz & Llinas, 1990): ביצועים חסינים (redundancy), כיסוי מרחבי וזמני מורחב, ביטחון מוגבר והפחתת עמימות, שיפור גילוי ורזולוציה, הגדלת ממד מרחב המדידה (עמידות לשיבוש).
בקצרה: מרחיב כיסוי ספקטרלי/מרחבי/זמני ומקטין אי-ודאות ועמימות; מקטין יחס אות-לרעש.

חסרונות / מלכודות: מיזוג נתונים תלויים (ספירה כפולה), מיזוג נתונים לא-עקביים, מיזוג נתון גרוע עם טוב, אומדן ממוזג חסר-משמעות, הסתמכות-יתר על 'המספר', ואי-הבנת ההנחות של האלגוריתם (רוב האלגוריתמים מניחים אי-תלות).

> **"סודות" ה-Data Fusion (D. Hall):** אין תחליף לחיישן טוב; עיבוד במורד הזרם אינו מתקן עיבוד גרוע במעלה הזרם; התוצאה הממוזגת עלולה להיות גרועה מהחיישן הטוב ביותר; אין אלגוריתמי קסם; לעולם לא יהיו מספיק נתוני אימון.

```
# הדגמה כמותית למלכודת: מיזוג שתי מדידות *תלויות* כאילו היו בלתי-תלויות
# בין שגיאות שני החיישנים. הנוסחה הנאיבית מתעלמת ממנה rho ננית קורלציה
rng = np.random.default_rng(0)
rho_values = np.linspace(0, 0.95, 20)
true_x = 0.0
s1 = s2 = 1.0  # סטיות תקן
naive_rmse, correct_rmse = [], []
for rho in rho_values:
    cov = np.array([[s1**2, rho*s1*s2],[rho*s1*s2, s2**2]])
    L = np.linalg.cholesky(cov)
    errs_naive, errs_corr = [], []
    for _ in range(4000):
```

```
        e = L @ rng.standard_normal(2)
        z1, z2 = true_x + e
        # נאיבי: ממוצע פשוט (מידוג שונות שווה) <- מתעלם מקורלציה
        x_naive = 0.5*z1 + 0.5*z2
        # נכון: BLUE עם מטריצת השונות המלאה
        ones = np.ones(2)
        w = np.linalg.solve(cov, ones); w /= ones @ w
        x_corr = w @ np.array([z1, z2])
        errs_naive.append((x_naive-true_x)**2)
        errs_corr.append((x_corr-true_x)**2)
    naive_rmse.append(np.sqrt(np.mean(errs_naive)))
    correct_rmse.append(np.sqrt(np.mean(errs_corr)))

plt.figure(figsize=(7,4))
plt.plot(rho_values, naive_rmse, 'o-', label='מידוג נאיבי (מתעלם מתלות)')
plt.plot(rho_values, correct_rmse, 's-', label='מידוג נכון (BLUE)')
plt.xlabel('בין שגיאות החיישנים ρ קורלציה'); plt.ylabel('RMSE של האומדן')
plt.title('מלכודת: מידוג נתונים תלויים כאילו בלתי-תלויים')
plt.legend(); plt.grid(alpha=.3); plt.tight_layout(); plt.show()
```



סיכום¶

- **Fusion = הקטנת אי-ודאות.** משוואת ה-Fusion (
$$1/P_F = 1/P_1 + 1/P_2$$
) היא הליבה — שקולה לעדכון מדידה בקלמן.
- **מודל JDL/DFIG** נותן טקסונומיה (L0–L5) מאות ועד האדם בלולאה.
- היתוך **מרחיב** כיסוי ו**מקטין** אי-ודאות — אך **אינו** יוצר יש מאין, ותלוי קריטית בהנחת אי-התלות.

המחברות הבאות יורדות לטכניקה: **אמידה (estimation)** — ליבת רמה L1.

02 · אמידת פרמטרים (Parameter Estimation)¶

מבוסס על: W. Dale Blair (GTRI), FUSION Boot Camp 2026 — Estimation in Information Fusion, חלק א'.

מתווה ההרצאה¶

❑ מבוא ❑ **אמידה פרמטרית (Parametric)** ❑ אמידת מצב סטוכסטית (Stochastic State) ❑ מעקב מטרות ❑ סיכום

ההבחנה המרכזית¶

- **פרמטר** — כמות **קבועה בזמן** אך לא ידועה (למשל מיקום מטרה נייחת, הטיית חיישן). אומדים אותה ממדידות רועשות.
- **מצב (state)** — כמות ש**מתפתחת בזמן** (מיקום/מהירות/תאוצה), בד"כ קשורה במשוואות דיפרנציאליות → נושא מחברת 03.

מודל המדידה הכללי:

$$z_j = h(x) + w_j$$

, ומבקשים אומד

$$\hat{x}(Z_k) \approx x$$

מתוך אוסף המדידות

$$Z_k = \{z_j\}_{j=1}^{k}$$

.

א. משפחות האומדים¶

אומד	הגדרה	מתי משתמשים
ML (Maximum Likelihood)	$\hat{x}_{ML} = \arg\max_x p(Z\|x)$	אין מידע מוקדם על x
MAP (Maximum A Posteriori)	$\hat{x}_{MAP} = \arg\max_x p(x\|Z) \propto p(Z\|x)\,p(x)$	יש prior $p(x)$
MMSE (Min. Mean Squared Error)	$\hat{x}_{MMSE} = E[x\|Z]$	מזעור $E\|x-\hat{x}\|^2$
LSE (Least Squares)	$\hat{x} = \arg\min_x \sum_j (z_j - h_j(x))^2$	אין מודל הסתברותי
WLSE (Weighted LSE)	$\hat{x} = \arg\min_x \sum_j (z_j - h_j(x))^2/\sigma_j^2$	שונויות מדידה שונות

עבור רעש גאוסיאני בלתי-תלוי, **ML ≡ WLSE** (משקלים = $1/\sigma_j^2$), וכאשר השונויות שוות **ML ≡ LSE**.

ב. LSE למערכת לינארית¶

כאשר

$z = Hx + w$

של batch)

N

מדידות), פתרון הריבועים-הפחותים הוא:

$$\hat{x} = (H^T H)^{-1} H^T z, \qquad P_{\hat{x}} = \sigma_w^2 (H^T H)^{-1}$$

ועבור WLSE עם מטריצת שונות

R

של המדידות:

$$\hat{x} = (H^T R^{-1} H)^{-1} H^T R^{-1} z, \qquad P_{\hat{x}} = (H^T R^{-1} H)^{-1}$$

דוגמת השקופית: 11 מדידות של מטרה נייחת עם

$\sigma_w = 2$

m. כאן

$h(x) = x$

, כלומר

$H = \mathbf{1}$

, והפתרון מצטמצם ל**ממוצע**:

$\hat{x} = \frac{1}{N}\sum z_j$

עם שונות

σ_w^2/N

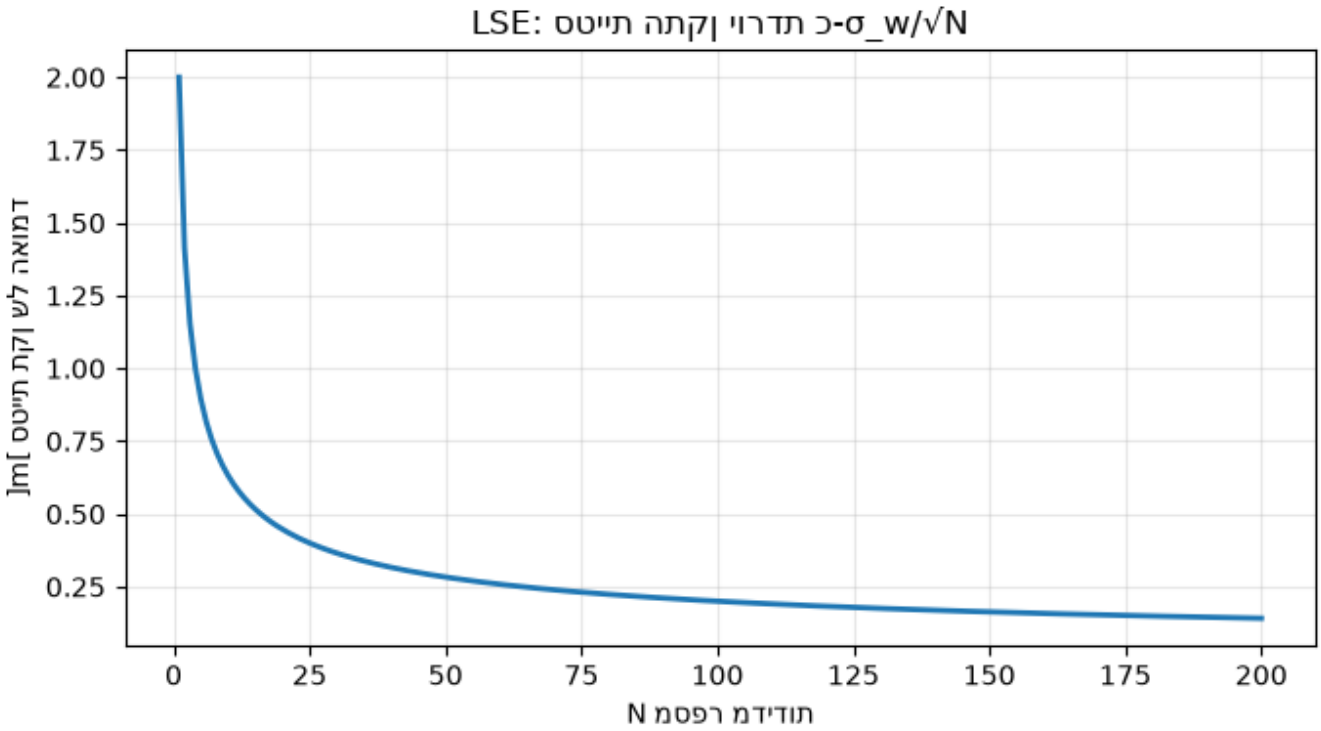
.

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(1)

def lse_linear(H, z, R=None):
    """LSE / WLSE למערכת לינארית z = H x + w. R = (אופציונלי) מטריצת שונות."""
    H = np.atleast_2d(H); z = np.asarray(z, float).reshape(-1, 1)
    if R is None:
        G = np.linalg.inv(H.T @ H) @ H.T
        xhat = G @ z
        return xhat.ravel(), np.linalg.inv(H.T @ H)   # P לקבלת sigma^2-להכפיל ב
    Ri = np.linalg.inv(R)
    P = np.linalg.inv(H.T @ Ri @ H)
    xhat = P @ H.T @ Ri @ z
    return xhat.ravel(), P

# דוגמת המטרה הנייחת (11 מדידות, sigma_w = 2)
true_x = 100.0; sigma_w = 2.0; N = 11
z = true_x + sigma_w * rng.standard_normal(N)
H = np.ones((N, 1))
xhat, M = lse_linear(H, z)
P = sigma_w**2 * M
print(f'אומד LSE = {xhat[0]:.3f} m   (אמת = {true_x})')
```

```
print(f'שונות האומד = {P[0,0]:.3f} = sigma_w^2/N = {sigma_w**2/N:.3f}')
print(f'סטיית תקן = {np.sqrt(P[0,0]):.3f} m')
```

```
אומד LSE = 100.414 m   (אמת = 100.0)
שונות האומד = sigma_w^2/N = 0.364
סטיית תקן = 0.603 m
```

```
# 1-כ סטיית התקן יורדת :התכנסות/sqrt(N)
Ns = np.arange(1, 201)
std_theory = sigma_w / np.sqrt(Ns)
plt.figure(figsize=(7,4))
plt.plot(Ns, std_theory, 'C0', lw=2)
plt.xlabel('מספר מדידות N'); plt.ylabel('סטיית תקן של האומד [m]')
plt.title('LSE: סטיית התקן יורדת כ-σ_w/√N'); plt.grid(alpha=.3)
plt.tight_layout(); plt.show()
```



ג. חסם Cramér–Rao התחתון (CRLB)¶

ה-CRLB הוא חסם תחתון על השונות של **כל** אומד חסר-הטיה:

$$\mathrm{Var}(\hat{x}) \geq \frac{1}{I(x)}, \qquad I(x) = -E\left[\frac{\partial^2 \ln p(Z|x)}{\partial x^2}\right]$$

כאשר

$I(x)$

היא **מידע פישר (Fisher Information)**. אומד שמשיג את החסם נקרא efficient. עבור

N

מדידות גאוסיאניות בלתי-תלויות של פרמטר נייח,

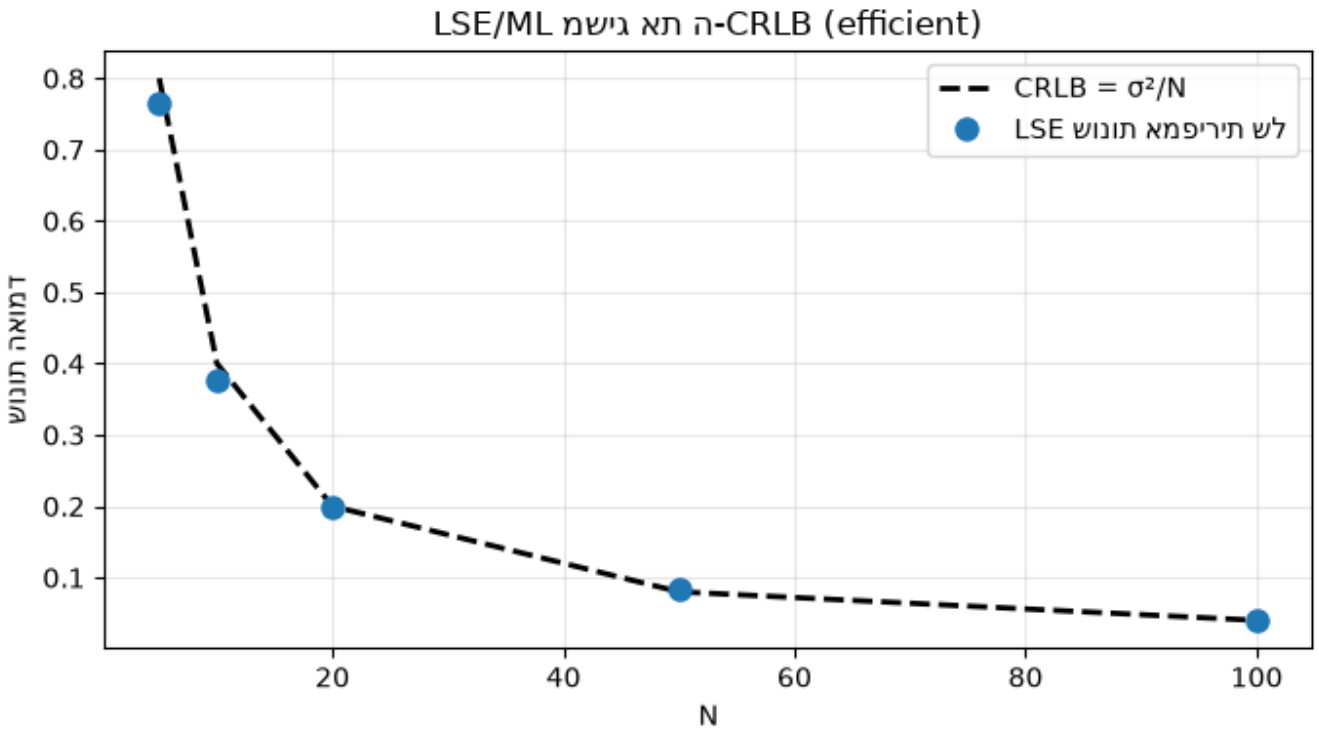
$I(x) = N/\sigma_w^2$

, ולכן

$$\mathrm{Var}(\hat{x}) \geq \sigma_w^2/N$$

— וה-LSE/ML משיג שוויון (efficient).

```
# אימות Monte-Carlo: האם LSE מגיע ל-CRLB?
sigma_w = 2.0
Ns = [5, 10, 20, 50, 100]
mc = 5000
emp_var, crlb = [], []
for N in Ns:
    ests = []
    for _ in range(mc):
        z = true_x + sigma_w*rng.standard_normal(N)
        ests.append(z.mean())
    emp_var.append(np.var(ests))
    crlb.append(sigma_w**2 / N)

plt.figure(figsize=(7,4))
plt.plot(Ns, crlb, 'k--', lw=2, label='CRLB = σ²/N')
plt.plot(Ns, emp_var, 'o', ms=8, label='שונות אמפירית של LSE')
plt.xlabel('N'); plt.ylabel('שונות האומד'); plt.legend(); plt.grid(alpha=.3)
plt.title('LSE/ML משיג את ה-CRLB (efficient)'); plt.tight_layout(); plt.show()
```



ד. אמידת ML — דוגמה לא-לינארית¶

Blair מדגים אמד ML על מודל לא-לינארי בסגנון

$$z_j = \cos(2\pi w x) + w_j$$

. כאן ה-likelihood אינו קמור ויש לחפש את המקסימום נומרית (סריקה / אופטימיזציה). להלן נשתמש במודל לא-לינארי פשוט יותר לאמידת **תדר**

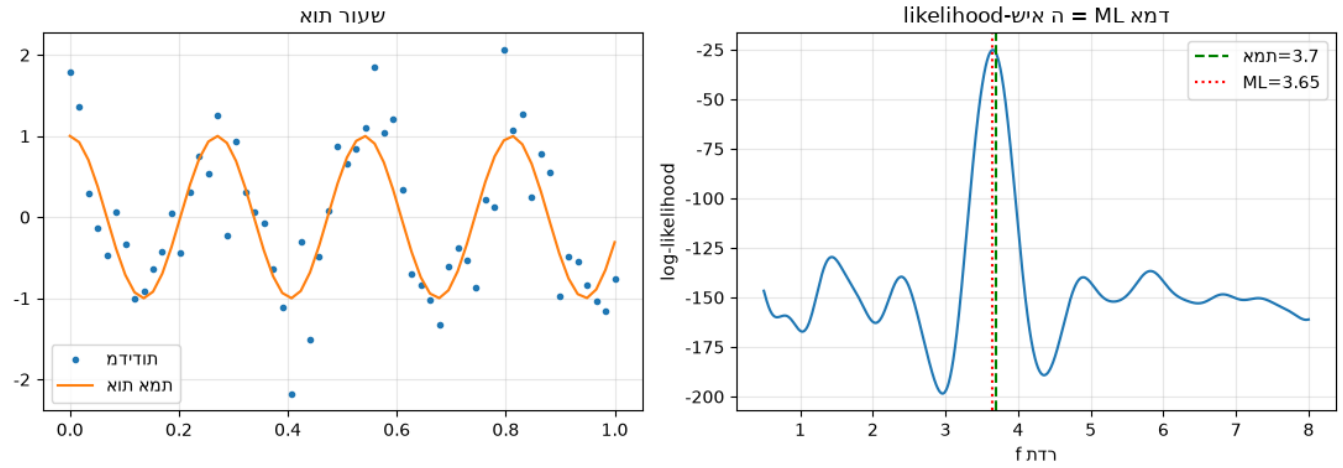
$$f$$

מתוך אות דגום:

$$z_j = \cos(2\pi f t_j) + w_j$$

.

```
# ML לאמידת תדר ע"י סריקת ה-log-likelihood
f_true = 3.7; sigma = 0.5
t = np.linspace(0, 1, 60)
z = np.cos(2*np.pi*f_true*t) + sigma*rng.standard_normal(t.size)

f_grid = np.linspace(0.5, 8, 1500)
# log-likelihood 0.5- :גאוסיאני/sigma^2 * sum (z - cos(2 pi f t))^2  (+const)
ll = np.array([-0.5/sigma**2 * np.sum((z - np.cos(2*np.pi*f*t))**2) for f in f_grid])
f_ml = f_grid[np.argmax(ll)]

fig, ax = plt.subplots(1, 2, figsize=(11,4))
ax[0].plot(t, z, '.', label='מדידות'); ax[0].plot(t, np.cos(2*np.pi*f_true*t), label='אות
אמת')
ax[0].set_title('אות רועש'); ax[0].legend(); ax[0].grid(alpha=.3)
ax[1].plot(f_grid, ll); ax[1].axvline(f_true, color='g', ls='--', label=f'אמת={f_true}')
ax[1].axvline(f_ml, color='r', ls=':', label=f'ML={f_ml:.2f}')
ax[1].set_xlabel('תדר f'); ax[1].set_ylabel('log-likelihood'); ax[1].set_title('אמד ML = שיא
ה-likelihood')
ax[1].legend(); ax[1].grid(alpha=.3); plt.tight_layout(); plt.show()
print(f'אומד ML של התדר = {f_ml:.3f}  (אמת = {f_true})')
```



אומד ML (אמת = 3.7) של התדר = 3.652

ה. אמידת הטיית חיישן (Sensor Bias) ובעיית התצפיתיות¶

Blair מקדיש כמה שקופיות לאמידת **הטיה (bias)** של חיישן. הלקח המרכזי: עם חיישן יחיד, המיקום

x

וההטיה

b

בלתי-נצפים בנפרד — המדידה היא תמיד

$$z = x + b + w$$

, וכל חלוקה בין

x

ל-

b

מתאימה לאותם נתונים. המטריצה

H^TH

נעשית סינגולרית ו"האומד אינו ניתן לחישוב".

הפתרון: הוספת מקור מידע נוסף (חיישן שני, או מטרת ייחוס/calibration שמיקומה ידוע) הופכת את הבעיה לנצפית — וזו בדיוק הנקודה שבה **fusion** נכנס: מיזוג מקורות פותר אי-תצפיתיות.

```
# הדגמה: חיישן יחיד -> H^T H סינגולרי (x ו-b בלתי נצפים בנפרד)
# מדידה: z = x + b + w. נעלמים: [x, b].
N = 10
H_single = np.hstack([np.ones((N,1)), np.ones((N,1))])  # שתי העמודות זהות!
print('דרגת H (חיישן יחיד):', np.linalg.matrix_rank(H_single), 'מתוך 2 <- סינגולרי, לא ניתן
להפריד x ו-b')

# מטרה לא ידועה (b לבדו נצפה) + פתרון: מטרת כיול במיקום ידוע x0=0
# חיישן מודד את מטרת הכיול ואת המטרה: z_cal = 0 + b + w ; z_tgt = x + b + w
x_true, b_true, sw = 50.0, 4.0, 1.0
m = 200
z_cal = 0 + b_true + sw*rng.standard_normal(m)
z_tgt = x_true + b_true + sw*rng.standard_normal(m)
# נעלמים theta=[x, b]; שתי משוואות: z_cal = b ; z_tgt = x + b
Hc = np.array([[0.,1.],[1.,1.]])
zc = np.array([z_cal.mean(), z_tgt.mean()])
theta, _ = lse_linear(Hc, zc, R=np.eye(2)*(sw**2/m))
print(f'עם מטרת כיול:  x̂ = {theta[0]:.2f} (אמת {x_true}),  b̂ = {theta[1]:.2f} (אמת {b_true})')
print('-> הפך את הבעיה לנצפית (fusion) מקור מידע נוסף.')
```

```
דרגת H (חיישן יחיד): 1 מתוך 2 <- סינגולרי, לא ניתן להפריד x ו-b
עם מטרת כיול:  x̂ = 50.00 (אמת 50.0),  b̂ = 3.90 (אמת 4.0)
-> הפך את הבעיה לנצפית (fusion) מקור מידע נוסף.
```

סיכום¶

- **ML / MAP / MMSE / LSE / WLSE** — משפחת אומדים; תחת רעש גאוסיאני הם מתלכדים (ML≡WLSE, ושווה-שונות ⇒ ≡LSE).
- **LSE לינארי:**

$$\hat{x} = (H^TR^{-1}H)^{-1}H^TR^{-1}z$$

עם שונות

$$(H^TR^{-1}H)^{-1}$$

.
- **CRLB** קובע את הרצפה לשונות; efficient LSE/ML עבור מטרה נייחת.
- **תצפיתיות:** הטיה ומיקום בלתי-נפרדים בחיישן יחיד — **fusion** של מקורות פותר זאת.

במחברת 03 נעבור מפרמטר **נייח** למצב ש**מתפתח בזמן** — וכך נגיע למסנן קלמן.

03 · אמידת מצב סטוכסטית ומסנן קלמן¶

מבוסס על: W. Dale Blair, FUSION Boot Camp 2026 — Estimation in Information Fusion, חלק ב'.

במחברת 02 אמדנו פרמטר **נייח**. כעת המצב **מתפתח בזמן** (מיקום, מהירות, תאוצה). **מסנן קלמן הוא 'סוס העבודה' של מעקב מטרות** ונותן את אומד ה-MMSE כאשר השגיאות גאוסיאניות.

א. המודל בזמן בדיד¶

$$x_{k+1} = F_k x_k + G_k v_k, \qquad z_k = H_k x_k + w_k$$

- x_k — וקטור המצב (מיקום, מהירות, ואולי תאוצה).
- $v_k \sim \mathcal{N}(0, Q_k)$ — **רעש תהליך** (process / plant noise).
- $w_k \sim \mathcal{N}(0, R_k)$ — רעש מדידה.
- F — מטריצת מעבר (אילוץ התנועה הקינמטי);
- H — קושר מדידות למצב (צורות לא-לינאריות → EKF, מחברת 04).

רעש התהליך הוא ההבדל המהותי בין מסנן קלמן (אמידת מצב סטוכסטית) לבין LSE (אמידה פרמטרית): הוא מבטא אי-ודאות בהתפתחות/מידול המצב מצעד k ל-$k+1$, אפס-תוחלת ולבן.

ב. גזירת מסנן קלמן — predictor / corrector¶

המסנן הוא אלגוריתם **חזאי–מתקן (predictor-corrector)**:

עדכון זמן (Time Update / Predict):

$$\hat{x}_{k|k-1} = F_{k-1}\hat{x}_{k-1|k-1}, \qquad P_{k|k-1} = F_{k-1}P_{k-1|k-1}F_{k-1}^T + G_{k-1}Q_{k-1}G_{k-1}^T$$

עדכון מדידה (Measurement Update / Correct):

$$S_k = H_k P_{k|k-1} H_k^T + R_k \quad \text{(השדחה תונוש / innovation)}$$

$$K_k = P_{k|k-1} H_k^T S_k^{-1} \quad \text{(Kalman gain)}$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k\,[\,z_k - H_k\hat{x}_{k|k-1}\,]$$

$$P_{k|k} = (I - K_k H_k)\,P_{k|k-1}$$

ה-gain

K_k

מאזן בין אמון במודל (predict) לאמון במדידה:

R

גדול →

K

קטן (מאמינים למודל), ולהפך.

אזהרת השקופית: מסנן קלמן מניח **שיוך מדידה-למסלול מושלם** (perfect measurement-to-track association) — זו בדיוק הנקודה שבה נכנס שיוך נתונים (הרצאת Meyer).

ג. מודל ה-NCV ורעש התהליך (DWNA מול CWNA)¶

Nearly Constant Velocity (NCV) — המודל הנפוץ ביותר במעקב. וקטור המצב

$x = [p, \dot{p}]^T$

:

$$F = \begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

שתי דרכים נפוצות למדל את רעש התהליך (תאוצה לא-מתוכננת בשונות

σ_a^2

או PSD

q

):

DWNA — Discrete White Noise Acceleration:

$$Q = \Gamma\Gamma^T\sigma_a^2, \quad \Gamma = \begin{bmatrix} T^2/2 \\ T \end{bmatrix} \Rightarrow Q = \sigma_a^2 \begin{bmatrix} T^4/4 & T^3/2 \\ T^3/2 & T^2 \end{bmatrix}$$

CWNA — Continuous White Noise Acceleration:

$$Q = q \begin{bmatrix} T^3/3 & T^2/2 \\ T^2/2 & T \end{bmatrix}$$

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(7)

def ncv_matrices(T, kind='DWNA', sigma_a=1.0):
    F = np.array([[1, T],[0, 1.0]])
    H = np.array([[1.0, 0.0]])
    if kind == 'DWNA':
        G = np.array([[T**2/2],[T]])
        Q = (G @ G.T) * sigma_a**2
    else:  # CWNA
        Q = sigma_a**2 * np.array([[T**3/3, T**2/2],[T**2/2, T]])
    return F, H, Q

class KalmanFilter:
    def __init__(self, F, H, Q, R, x0, P0):
        self.F,self.H,self.Q,self.R = map(np.atleast_2d,(F,H,Q,R))
        self.x = np.asarray(x0,float).reshape(-1,1)
        self.P = np.atleast_2d(P0).astype(float)
    def predict(self):
        self.x = self.F @ self.x
        self.P = self.F @ self.P @ self.F.T + self.Q
        return self.x.ravel()
    def update(self, z):
        z = np.atleast_2d(np.asarray(z,float).reshape(-1,1))
        S = self.H @ self.P @ self.H.T + self.R
        K = self.P @ self.H.T @ np.linalg.inv(S)
        y = z - self.H @ self.x
        self.x = self.x + K @ y
        I = np.eye(self.P.shape[0])
        self.P = (I - K @ self.H) @ self.P
        return self.x.ravel(), y.item(), S.item()

print('Kalman filter + NCV models defined.')
```

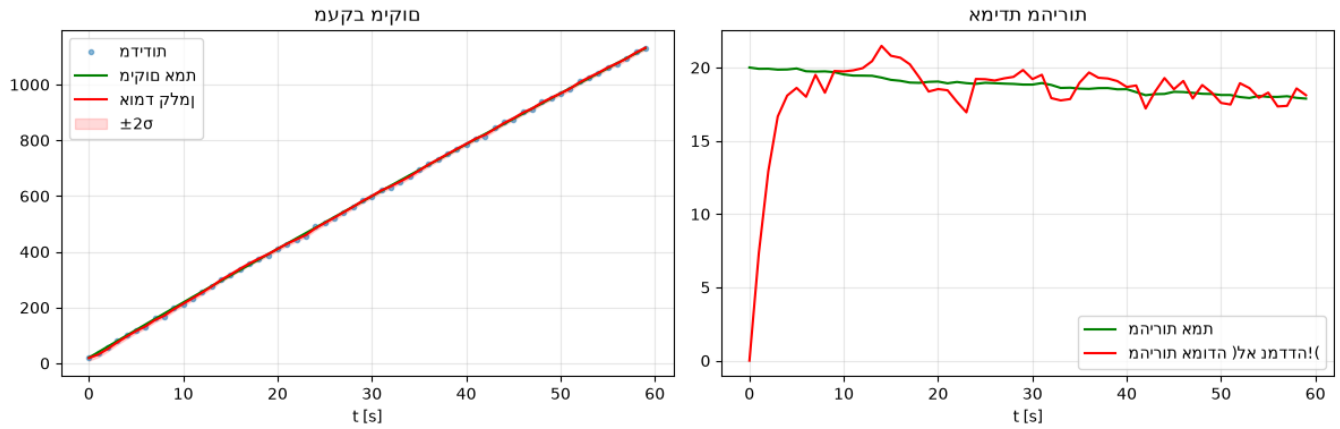
```
Kalman filter + NCV models defined.
```

ד. סימולציה: מעקב NCV אחר מטרה במהירות קבועה¶

מטרה נעה במהירות קבועה; החיישן מודד **רק מיקום** עם רעש σ_w. המסנן אומד מיקום ו**מהירות** (שאינה נמדדת ישירות) — זהו כוחה של אמידת המצב.

```
T, steps = 1.0, 60
sigma_w = 5.0          # רעש מדידה [m]
sigma_a_true = 0.1     # תמרון אמיתי קטן
F, H, _ = ncv_matrices(T)
```

```
Qsim = ncv_matrices(T, 'DWNA', sigma_a_true)[2]

# יצירת מסלול אמת
x = np.array([[0.],[20.]])   # 20 מהירות ,0 מיקום m/s
truth, meas = [], []
Lq = np.linalg.cholesky(Qsim + 1e-12*np.eye(2))
for _ in range(steps):
    x = F @ x + Lq @ rng.standard_normal((2,1))
    truth.append(x.ravel().copy())
    meas.append((H @ x).item() + sigma_w*rng.standard_normal())
truth = np.array(truth); meas = np.array(meas)

# הרצת המסנן
Fk, Hk, Qk = ncv_matrices(T, 'DWNA', sigma_a=1.0)
kf = KalmanFilter(Fk, Hk, Qk, R=sigma_w**2, x0=[meas[0],0], P0=np.diag([sigma_w**2,100]))
est_pos, est_vel, pos_std = [], [], []
for z in meas:
    kf.predict()
    xe,_,_ = kf.update(z)
    est_pos.append(xe[0]); est_vel.append(xe[1]); pos_std.append(np.sqrt(kf.P[0,0]))
est_pos,est_vel,pos_std = map(np.array,(est_pos,est_vel,pos_std))

t = np.arange(steps)*T
fig, ax = plt.subplots(1,2,figsize=(12,4))
ax[0].plot(t, meas,'.',alpha=.5,label='מדידות')
ax[0].plot(t, truth[:,0],'g',label='מיקום אמת')
ax[0].plot(t, est_pos,'r',label='אומד קלמן')
ax[0].fill_between(t, est_pos-2*pos_std, est_pos+2*pos_std, color='r', alpha=.15, label='±2σ')
ax[0].set_title('מעקב מיקום'); ax[0].legend(); ax[0].grid(alpha=.3); ax[0].set_xlabel('t [s]')
ax[1].plot(t, truth[:,1],'g',label='מהירות אמת')
ax[1].plot(t, est_vel,'r',label='מהירות אמודה (לא נמדדה!)')
ax[1].set_title('אמידת מהירות'); ax[1].legend(); ax[1].grid(alpha=.3); ax[1].set_xlabel('t
[s]')
plt.tight_layout(); plt.show()
print(f'RMSE גולמי :מיקום={np.sqrt(np.mean((meas-truth[:,0])**2)):.2f} m,'
      f' מסונן={np.sqrt(np.mean((est_pos-truth[:,0])**2)):.2f} m')
```



RMSE גולמי: מיקום=4.35 m, מסונן=3.41 m

ה. תרומת רעש התהליך והתכנסות ה-gain¶

כאשר

Q

קבוע ו-

R

קבוע, מטריצת השונות

P

וה-gain

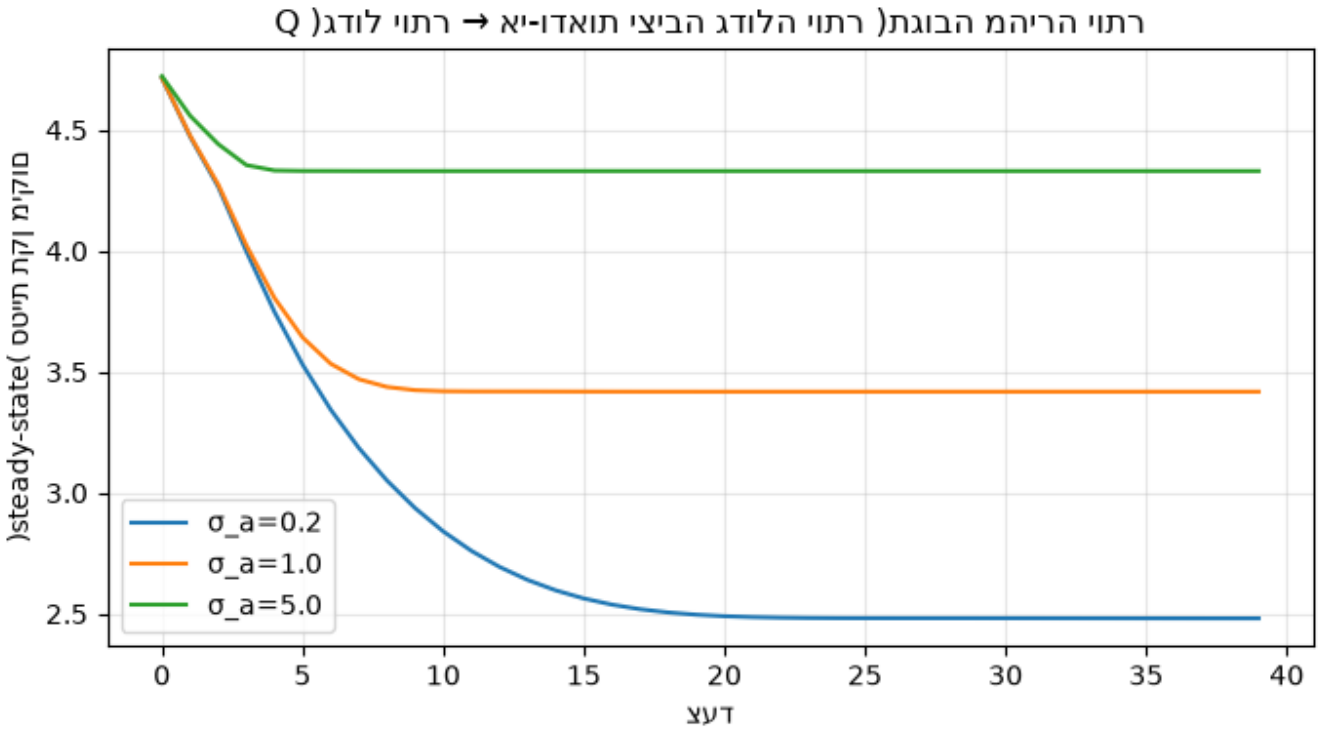
K

מתכנסים למצב יציב (steady state). ככל ש-

Q

קטן יותר — המסנן 'מאמין' יותר למודל, ה-gain קטן והאומד חלק אך איטי לתמרונים (נושא מחברת 04).

```
# שונים sigma_a עבור ערכי position std- ו gain התכנסות
plt.figure(figsize=(7,4))
for sa in [0.2, 1.0, 5.0]:
    Fk,Hk,Qk = ncv_matrices(T,'DWNA',sigma_a=sa)
    kf = KalmanFilter(Fk,Hk,Qk,R=sigma_w**2,x0=[0,0],P0=np.diag([100.,100.]))
    stds=[]
    for _ in range(40):
        kf.predict(); kf.update(0.0); stds.append(np.sqrt(kf.P[0,0]))
    plt.plot(stds, label=f'σ_a={sa}')
plt.xlabel('צעד'); plt.ylabel('סטיית תקן מיקום (steady-state)')
plt.title('Q (תגובה מהירה יותר) אי-ודאות יציבה גדולה יותר ← גדול יותר')
plt.legend(); plt.grid(alpha=.3); plt.tight_layout(); plt.show()
```



ו. בדיקת עקביות (NEES / innovation)¶

מסנן 'עקבי' (consistent) מייצר חדשות (innovations) שמתאימות לשונות

S_k

שהוא עצמו מנבא. מדד נפוץ: ה-innovation המנורמל

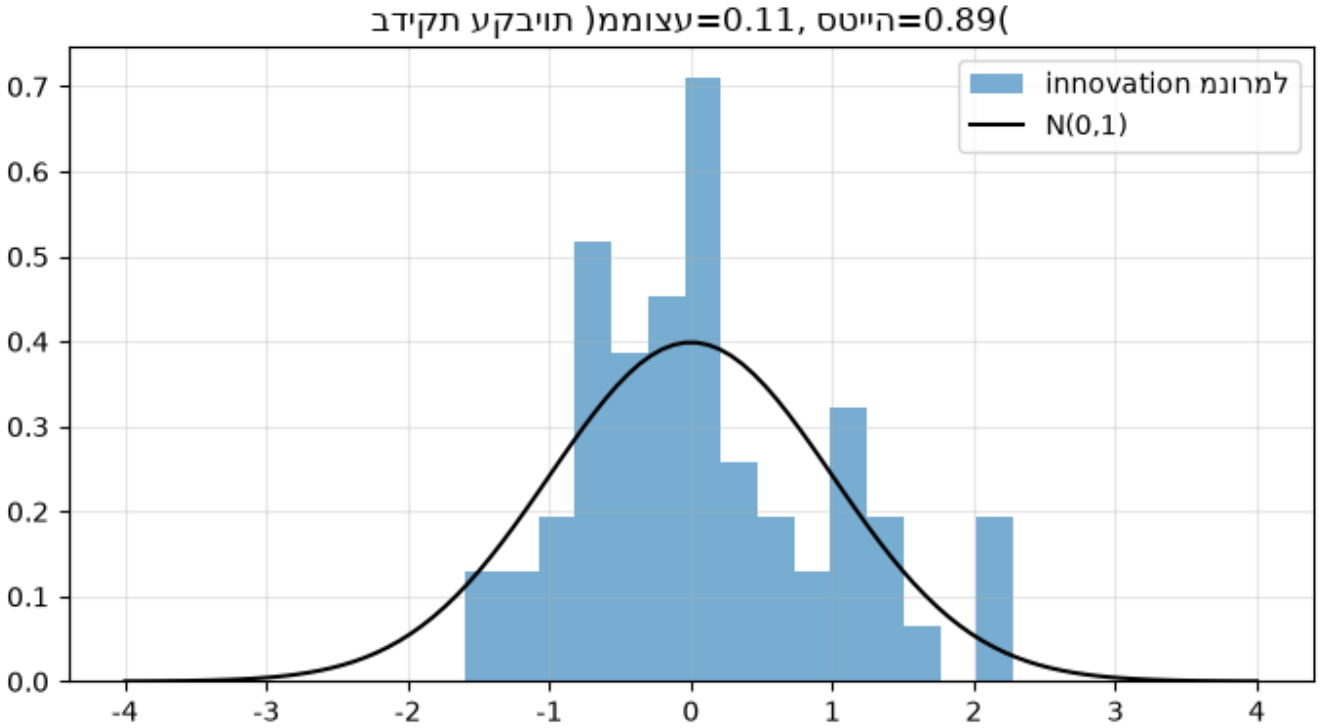
$y_k/\sqrt{S_k}$

אמור להתפלג כ-

$\mathcal{N}(0,1)$

.

```
kf = KalmanFilter(Fk:=ncv_matrices(T,'DWNA',1.0)[0], ncv_matrices(T)[1],
                  ncv_matrices(T,'DWNA',1.0)[2], R=sigma_w**2,
                  x0=[meas[0],0], P0=np.diag([sigma_w**2,100]))
norm_innov=[]
for z in meas:
    kf.predict(); _,y,S = kf.update(z); norm_innov.append(y/np.sqrt(S))
norm_innov=np.array(norm_innov)
plt.figure(figsize=(7,4))
plt.hist(norm_innov, bins=15, density=True, alpha=.6, label='innovation מנורמל')
xx=np.linspace(-4,4,100); plt.plot(xx, np.exp(-xx**2/2)/np.sqrt(2*np.pi),'k',label='N(0,1)')
plt.title(f'ממוצע) תוביקע תקידב=0.11, הייטס=0.89(')
plt.legend(); plt.grid(alpha=.3); plt.tight_layout(); plt.show()
```



סיכום¶

- **מסנן קלמן = predictor–corrector**; נותן MMSE תחת לינאריות+גאוסיאניות.
- **רעש התהליך** (

Q

, בצורת DWNA/CWNA) מבדיל אמידת מצב מאמידה פרמטרית, ושולט באיזון מודל↔מדידה.
- מודל **NCV** אומד מהירות גם כשמודדים רק מיקום, ומקטין משמעותית את שגיאת המיקום מול המדידה הגולמית.
- המסנן מניח **שיוך מדידה מושלם** ומודל **יחיד** — שתי הנחות שמחברת 04 מרפה (תמרון, IMM, אי-לינאריות).

04 · מעקב אחר מטרות מתמרנות: NCV/NCA, IMM ו-EKF¶

מבוסס על: W. Dale Blair, FUSION Boot Camp 2026 — Estimation in Information Fusion, חלק ג'.

במחברת 03 בנינו מסנן קלמן עם מודל תנועה **יחיד** (NCV). מטרות אמיתיות **מתמרנות**, וכאן עולות שלוש סוגיות: בחירת מודל (NCV מול NCA), כיול רעש התהליך, ושילוב מספר מודלים (IMM). בנוסף, מדידות אמיתיות (טווח/זווית) הן **לא-לינאריות** → EKF.

א. NCV מול NCA¶

- **NCV** (Nearly Constant Velocity): מצב $[p, \dot{p}]$. פשוט; מצוין לתנועה ישרה; **מפגר בתמרון**.
- **NCA** (Nearly Constant Acceleration): מצב $[p, \dot{p}, \ddot{p}]$. עוקב אחרי תאוצה; אך 'רועש' יותר בתנועה שקטה (מאמד תאוצה מרעש).

מטריצות NCA (DWNA):

$$F = \begin{bmatrix} 1 & T & T^2/2 \\ 0 & 1 & T \\ 0 & 0 & 1 \end{bmatrix},\ H = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix},\ Q = \sigma_a^2\,\Gamma\Gamma^T,\ \Gamma = \begin{bmatrix} T^2/2 \\ T \\ 1 \end{bmatrix}$$

הדילמה (מהשקופיות):

σ_a קטן → שגיאות קטנות בתנועה שקטה אך פיגור גדול בתמרון;

σ_a גדול → ממזער את השגיאה המרבית בתמרון אך מגדיל את השגיאה הממוצעת. **אין בחירה אחת אופטימלית** — וזו המוטיבציה ל-IMM.

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(11)

def ncv(T, sa):
    F=np.array([[1,T],[0,1.]]); H=np.array([[1.,0]])
    G=np.array([[T**2/2],[T]]); Q=(G@G.T)*sa**2
    return F,H,Q
def nca(T, sa):
    F=np.array([[1,T,T**2/2],[0,1,T],[0,0,1.]]); H=np.array([[1.,0,0]])
    G=np.array([[T**2/2],[T],[1.]]); Q=(G@G.T)*sa**2
    return F,H,Q

class KF:
```

```
    def __init__(s,F,H,Q,R,x,P): s.F,s.H,s.Q,s.R=map(np.atleast_2d,(F,H,Q,R));
s.x=np.array(x,float).reshape(-1,1); s.P=np.array(P,float)
    def predict(s): s.x=s.F@s.x; s.P=s.F@s.P@s.F.T+s.Q
    def update(s,z):
        S=s.H@s.P@s.H.T+s.R; K=s.P@s.H.T@np.linalg.inv(S)
        y=np.atleast_2d(z).reshape(-1,1)-s.H@s.x
        s.x=s.x+K@y; s.P=(np.eye(s.P.shape[0])-K@s.H)@s.P
        d=float((y.T@np.linalg.inv(S)@y).item())
        L=float(np.exp(-0.5*d)/np.sqrt(np.linalg.det(2*np.pi*S)))
        return L
print('NCV/NCA + KF defined.')
```

```
NCV/NCA + KF defined.
```

```
# 40-60 תאוצה <- מסלול אמת עם תמרון: מהירות קבועה -> שוב קבועה
T=1.0; N=100; sw=10.0
t=np.arange(N)*T
acc=np.where((t>=40)&(t<60), 8.0, 0.0)   # תאוצה m/s^2 בקטע התמרון
pos=np.zeros(N); vel=np.zeros(N); vel[0]=30
for k in range(1,N):
    vel[k]=vel[k-1]+acc[k-1]*T
    pos[k]=pos[k-1]+vel[k-1]*T+0.5*acc[k-1]*T**2
meas=pos+sw*rng.standard_normal(N)

def run_single(make, sa, dim):
    F,H,Q=make(T,sa); P0=np.diag([sw**2,100,100][:dim])
    x0=[meas[0]]+[0]*(dim-1)
    kf=KF(F,H,Q,sw**2,x0,P0); est=[]
    for z in meas: kf.predict(); kf.update(z); est.append(kf.x[0,0])
    return np.array(est)

est_ncv_lo=run_single(ncv,0.5,2)
est_ncv_hi=run_single(ncv,8.0,2)
est_nca   =run_single(nca,8.0,3)
for name,e in [('NCV σ_a=0.5',est_ncv_lo),('NCV σ_a=8',est_ncv_hi),('NCA σ_a=8',est_nca)]:
    print(f'{name:14s} RMSE={np.sqrt(np.mean((e-pos)**2)):6.2f} m   '
          f'RMSE בתמרון={np.sqrt(np.mean((e[40:60]-pos[40:60])**2)):6.2f} m')
```

```
NCV σ_a=0.5    RMSE= 55.84 m   RMSE בתמרון=100.67 m
NCV σ_a=8      RMSE=  7.26 m   RMSE בתמרון=  9.29 m
NCA σ_a=8      RMSE=  7.86 m   RMSE בתמרון=  8.11 m
```

התוצאה ממחישה את הדילמה: NCV עם

σ_a

קטן מצוין בכללי אך גרוע בתמרון; הגדלת

σ_a

או מעבר ל-NCA משפר את התמרון על חשבון רעש בתנועה השקטה.

ב. אומד IMM (Interacting Multiple Model)¶

ה-IMM מריץ **כמה מסננים במקביל** (למשל NCV שקט + NCV/NCA מתמרן) ומשקלל אותם לפי **הסתברות מודל** המתעדכנת מה-likelihood של כל מודל. מחזור IMM (לפי השקופית 'IMM Algorithm with Two Models'):

1. **Interaction/Mixing** — ערבוב אומדני המודלים לפי מטריצת המעבר π_{ij} והסתברויות μ.
2. **Filtering** — כל מודל מבצע predict+update מהאומד המעורבב, ומחשב likelihood Λ_j.
3. **Model Probability Update** — $\mu_j \propto \Lambda_j \sum_i \pi_{ij} \mu_i$.
4. **Combination** — אומד פלט = שקלול האומדנים בהסתברויות μ_j.

המפתח (מהשקופית): "המפתח להיתוך מוצלח הוא הגדלת סטיית התקן של המסנן בזמן התמרון" — וזה בדיוק מה ש-IMM עושה אוטומטית.

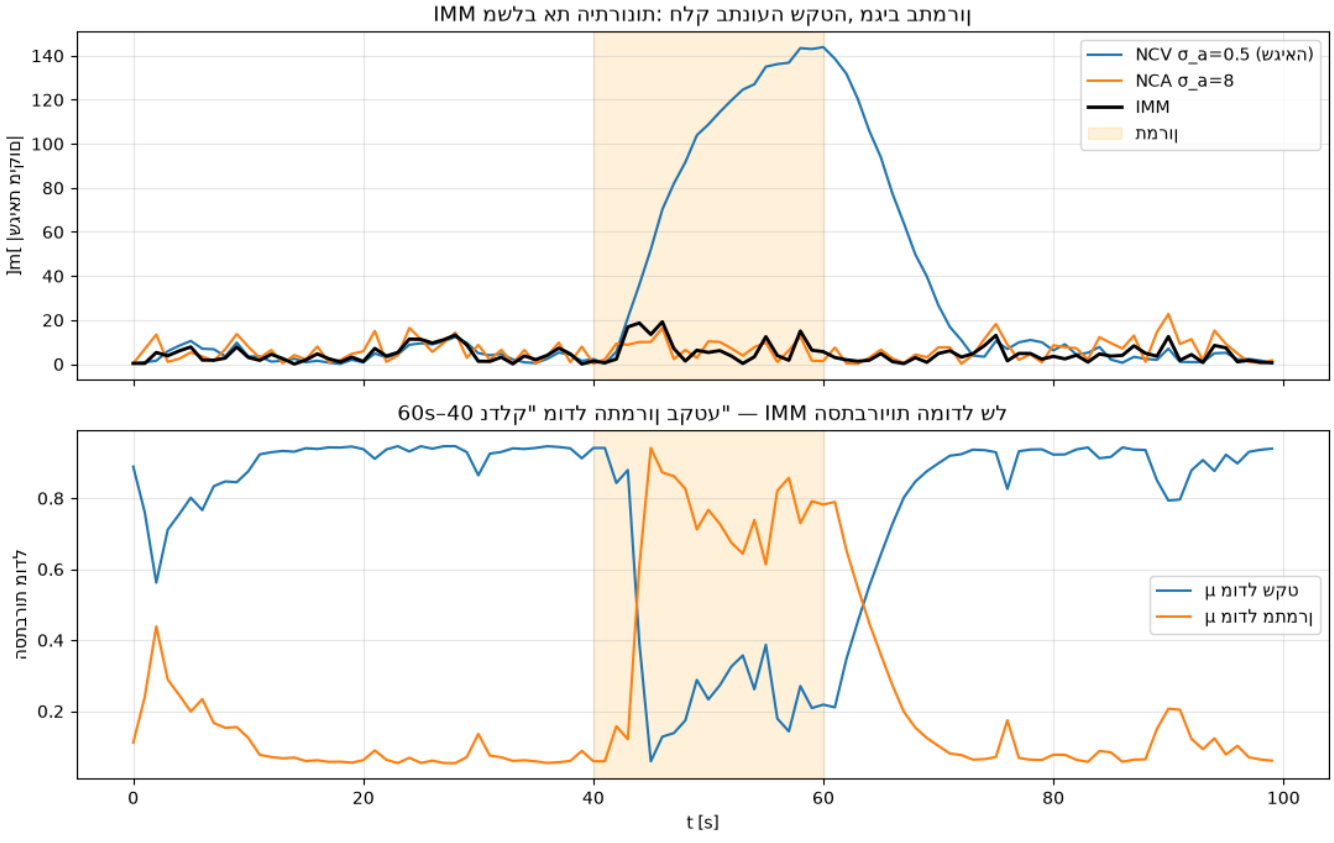
```
class IMM:
    """IMM עם מספר מסננים בעלי ממד מצב זהה."""
    def __init__(self, filters, Pi, mu):
        self.f=filters; self.Pi=np.array(Pi,float); self.mu=np.array(mu,float)
    def step(self, z):
        r=len(self.f)
        cbar=self.Pi.T@self.mu                      # הסתברויות מודל חזויות
        mix=(self.Pi*self.mu[:,None])/cbar[None,:]  # mu_{i|j}
        xs=[f.x.copy() for f in self.f]; Ps=[f.P.copy() for f in self.f]
        # ערבוב
        for j in range(r):
            x0=sum(mix[i,j]*xs[i] for i in range(r))
            P0=sum(mix[i,j]*(Ps[i]+(xs[i]-x0)@(xs[i]-x0).T) for i in range(r))
            self.f[j].x=x0; self.f[j].P=P0
        # סינון + likelihood
        L=np.zeros(r)
        for j in range(r):
            self.f[j].predict(); L[j]=self.f[j].update(z)
        # עדכון הסתברות מודל
        self.mu=cbar*L; self.mu/= self.mu.sum() if self.mu.sum()>0 else 1
        # שקלול פלט
        x=sum(self.mu[j]*self.f[j].x for j in range(r))
        P=sum(self.mu[j]*(self.f[j].P+(self.f[j].x-x)@(self.f[j].x-x).T) for j in range(r))
        return x.ravel(), P, self.mu.copy()
print('IMM defined.')
```

```
IMM defined.
```

```
# IMM עם שני מודלי NCV: שקט (σ_a קטן) ומתמרן (σ_a גדול)
f_quiet=KF(*ncv(T,0.3), sw**2, [meas[0],0], np.diag([sw**2,100.]))
f_maneu=KF(*ncv(T,12.0),sw**2, [meas[0],0], np.diag([sw**2,100.]))
Pi=[[0.97,0.03],[0.10,0.90]]   # מטריצת מעבר בין מצבים
imm=IMM([f_quiet,f_maneu], Pi, [0.9,0.1])

est_imm=[]; mu_hist=[]
for z in meas:
    x,P,mu=imm.step(z); est_imm.append(x[0]); mu_hist.append(mu)
est_imm=np.array(est_imm); mu_hist=np.array(mu_hist)
print(f'IMM RMSE כללי={np.sqrt(np.mean((est_imm-pos)**2)):.2f} m,'
      f' בתמרון={np.sqrt(np.mean((est_imm[40:60]-pos[40:60])**2)):.2f} m')
```

IMM RMSE כללי=6.33 m, בתמרון=9.51 m

```
fig,ax=plt.subplots(2,1,figsize=(11,7),sharex=True)
ax[0].plot(t,np.abs(est_ncv_lo-pos),label='NCV σ_a=0.5 (שגיאה)')
ax[0].plot(t,np.abs(est_nca-pos),label='NCA σ_a=8')
ax[0].plot(t,np.abs(est_imm-pos),'k',lw=2,label='IMM')
ax[0].axvspan(40,60,color='orange',alpha=.15,label='תמרון')
ax[0].set_ylabel('|שגיאת מיקום| [m]'); ax[0].legend(); ax[0].grid(alpha=.3)
ax[0].set_title('IMM מגיב בתמרון, חלק בתנועה שקטה: משלב את היתרונות')
ax[1].plot(t,mu_hist[:,0],label='μ מודל שקט')
ax[1].plot(t,mu_hist[:,1],label='μ מודל מתמרן')
ax[1].axvspan(40,60,color='orange',alpha=.15)
ax[1].set_xlabel('t [s]'); ax[1].set_ylabel('הסתברות מודל'); ax[1].legend();
ax[1].grid(alpha=.3)
ax[1].set_title('הסתברויות המודל של IMM — "60–40 מודל התמרון בקטע "קלדנs')
plt.tight_layout(); plt.show()
```



ג. ריבוי חיישנים: האם יותר נתונים תמיד עדיף?¶

Blair מציג ניסוי ("Does More Data Always Mean Better...?"): בזמן תמרון, **יותר מדידות בשונות-תהליך לא מתאימה עלולות להזיק**. עם זאת, "יותר נתונים אכן מובילים לאומד טוב יותר כשההיתוך תחת ההשערה הנכונה (מתמרן)" — ולכן IMM, שמגדיל את אי-הוודאות בזמן תמרון, הוא המפתח לניצול נכון של נתונים נוספים.

```
# IMM עם (2 Hz) מול שני חיישנים מנותבים (1 Hz) השוואה: חיישן יחיד
def simulate_imm(meas_t, meas_z, dt):
    fq=KF(*ncv(dt,0.3),sw**2,[meas_z[0],0],np.diag([sw**2,100.]))
    fm=KF(*ncv(dt,12.0),sw**2,[meas_z[0],0],np.diag([sw**2,100.]))
    im=IMM([fq,fm],Pi,[0.9,0.1]); out=[]
    for z in meas_z: x,_,_=im.step(z); out.append(x[0])
    return np.array(out)

# 2 Hz: (מדמה חיישן שני מנותב) דגימה כפולה
t2=np.arange(0,N,0.5); idx=np.minimum((t2/T).astype(int),N-1)
meas2=pos[idx]+sw*rng.standard_normal(t2.size)
est2=simulate_imm(t2,meas2,0.5)
# 1-ל Hz דגימה חזרה להשוואה
est2_at1=est2[::2][:N]
print(f'IMM 1 Hz  RMSE בתמרון = {np.sqrt(np.mean((est_imm[40:60]-pos[40:60])**2)):.2f} m')
print(f'IMM 2 Hz  RMSE בתמרון = {np.sqrt(np.mean((est2_at1[40:60]-pos[40:60])**2)):.2f} m')
print('-> עם המודל הנכון (IMM), יותר נתונים אכן משפרים את האומד בתמרון.')
```

```
IMM 1 Hz  RMSE בתמרון = 9.51 m
IMM 2 Hz  RMSE בתמרון = 24.64 m
-> יותר נתונים אכן משפרים את האומד בתמרון, עם המודל הנכון (IMM).
```

ד. מסנן קלמן מורחב (EKF) למדידות לא-לינאריות¶

מדידות אמיתיות (רדאר) הן **טווח וזווית**,

$$z = h(x) + w$$

עם

h

לא-לינארי. ה-EKF מליניאריזציה סביב האומד:

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k[z_k - h(\hat{x}_{k|k-1})], \quad K_k = P_{k|k-1}H_k^T S_k^{-1}, \quad H_k = \frac{\partial h}{\partial x}\Big|_{\hat{x}_{k|k-1}}$$

כאשר

H_k

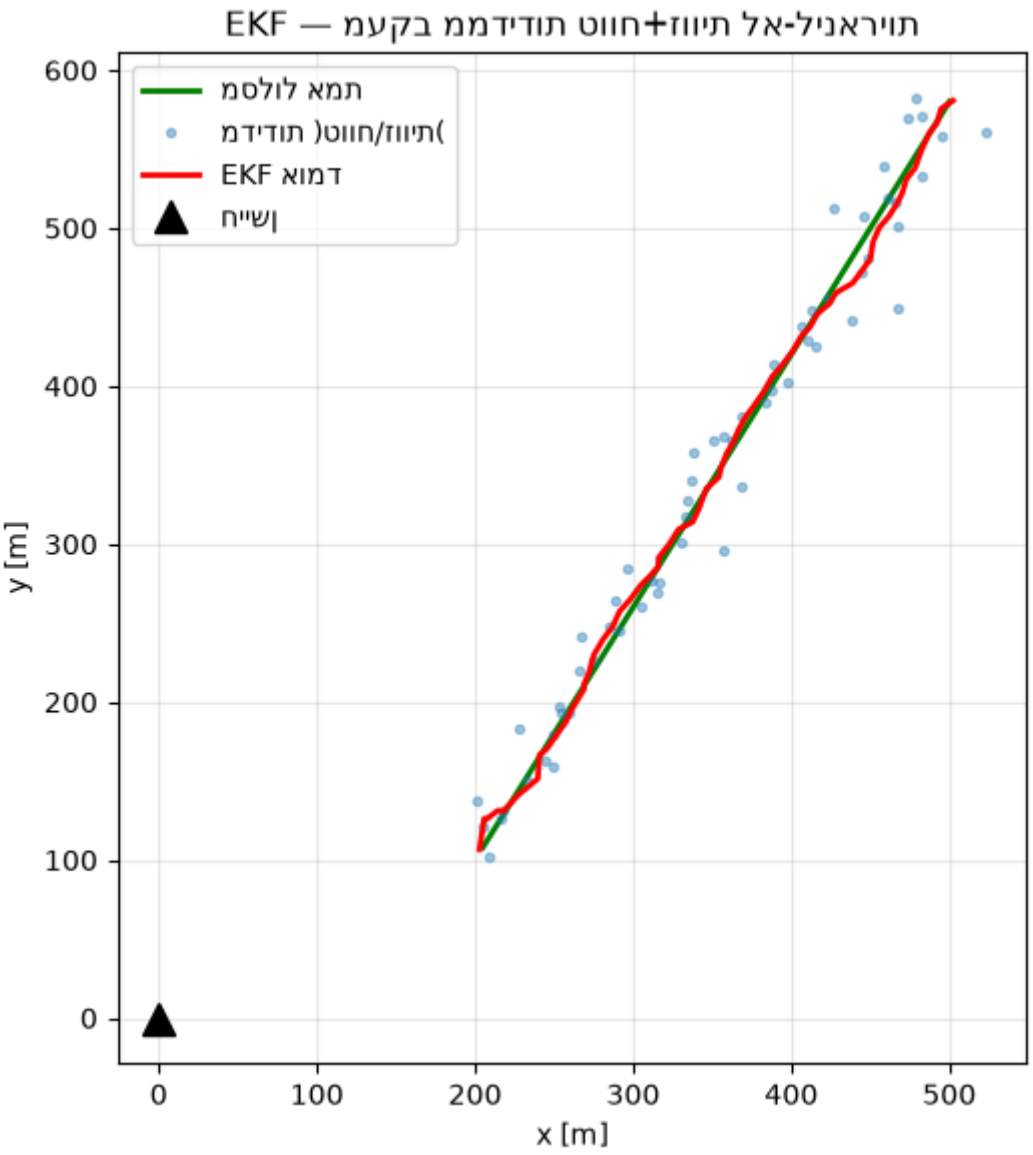
היא ה**יעקוביאן** של

h

. נדגים מעקב 2D ממדידות טווח+זווית מחיישן בראשית.

```
# EKF: מצב [x, vx, y, vy], מדידה = [range, bearing]
dt=1.0; M=60
Fb=np.array([[1,dt,0,0],[0,1,0,0],[0,0,1,dt],[0,0,0,1.]])
q=0.05
Qb=q*np.array([[dt**3/3,dt**2/2,0,0],[dt**2/2,dt,0,0],
               [0,0,dt**3/3,dt**2/2],[0,0,dt**2/2,dt]])
```

```
Rb=np.diag([8.0**2, np.deg2rad(1.5)**2])   # שונות טווח [m], זווית [rad]

def h(x):
    r=np.hypot(x[0],x[2]); b=np.arctan2(x[2],x[0]); return np.array([r,b])
def Hjac(x):
    px,py=x[0],x[2]; r2=px**2+py**2; r=np.sqrt(r2)
    return np.array([[px/r,0,py/r,0],[-py/r2,0,px/r2,0]])

# מסלול אמת
xt=np.array([200.,5.,100.,8.]); truth=[]; zs=[]
for _ in range(M):
    xt=Fb@xt; truth.append(xt.copy())
    z=h(xt)+np.array([8.0*rng.standard_normal(), np.deg2rad(1.5)*rng.standard_normal()])
    zs.append(z)
truth=np.array(truth); zs=np.array(zs)

# EKF
x=np.array([180.,0.,120.,0.]); P=np.diag([100,50,100,50.]); est=[]
for z in zs:
    x=Fb@x; P=Fb@P@Fb.T+Qb                        # predict
    H=Hjac(x); S=H@P@H.T+Rb; K=P@H.T@np.linalg.inv(S)
    y=z-h(x); y[1]=(y[1]+np.pi)%(2*np.pi)-np.pi    # עטיפת זווית
    x=x+K@y; P=(np.eye(4)-K@H)@P                    # update
    est.append(x.copy())
est=np.array(est)

# מדידות גולמיות בקואורדינטות קרטזיות
mx=zs[:,0]*np.cos(zs[:,1]); my=zs[:,0]*np.sin(zs[:,1])
plt.figure(figsize=(7,6))
plt.plot(truth[:,0],truth[:,2],'g',lw=2,label='מסלול אמת')
plt.plot(mx,my,'.',alpha=.4,label='מדידות (טווח/זווית)')
plt.plot(est[:,0],est[:,2],'r',lw=2,label='EKF אומד')
plt.plot(0,0,'k^',ms=12,label='חיישן'); plt.gca().set_aspect('equal')
plt.xlabel('x [m]'); plt.ylabel('y [m]'); plt.legend(); plt.grid(alpha=.3)
plt.title('EKF — מעקב ממדידות טווח+זווית לא-לינאריות'); plt.tight_layout(); plt.show()
print(f'EKF RMSE מיקום = {np.sqrt(np.mean((est[:,0]-truth[:,0])**2+(est[:,2]-truth[:,
2])**2)):.2f} m')
```



EKF RMSE מיקום = 5.00 m

סיכום¶

- **NCV מול NCA:** אין מודל יחיד אופטימלי — קטן ב-
σ_a
טוב בתנועה שקטה וגרוע בתמרון, ולהפך.
- **IMM** מריץ מודלים במקביל ומשקלל לפי הסתברות מודל; משיג את 'הטוב משני העולמות' ומגדיל אוטומטית את אי-הוודאות בתמרון.
- יותר נתונים עוזרים **רק תחת ההשערה/מודל הנכון** — זו המוטיבציה ל-IMM ולבחירת
Q
מושכלת.
- **EKF** מטפל במדידות לא-לינאריות (טווח/זווית) דרך ליניאריזציה ביעקוביאן
H_k
.

בכך מסתיימת הסדרה המכסה את שני המערכים שב-PDF (Blasch — מבוא, Blair — אמידה). ההרצאות 3–7 (שיוך נתונים, מעקב מבוזר, סיווג, היסק מבוזר, fusion ברמה גבוהה) ממשיכות ישירות מהיסודות כאן — ראו מחברת 00 למיפוי.

05 · שיוך נתונים (Data Association)¶

מלווה את ההרצאה: Florian Meyer, FUSION Boot Camp 2026 — Data Association.

שקופיות הרצאה זו **אינן** כלולות ב-PDF. המחברת מבוססת על הספרות הקנונית (Bar-Shalom, Blackman & Popoli) ועל ההקשר שמספק Blasch במבוא.

במחברת 03 ראינו שמסנן קלמן מניח **שיוך מדידה-למסלול מושלם**. במציאות יש מספר מטרות, מדידות שווא (clutter), והחמצות (missed detections). שיוך נתונים עונה על השאלה: **איזו מדידה שייכת לאיזה מסלול?**

א. שערור (Gating) ומרחק מהלנוביס¶

לפני שיוך, מסננים מדידות שרחוקות מדי מהחיזוי. ה'שער' (gate) מוגדר ע"י מרחק מהלנוביס בריבוע:

$$d^2 = (z - \hat{z})^T S^{-1} (z - \hat{z}) \le \gamma$$

כאשר

$$S = HPH^T + R$$

היא שונות החדשה (innovation).

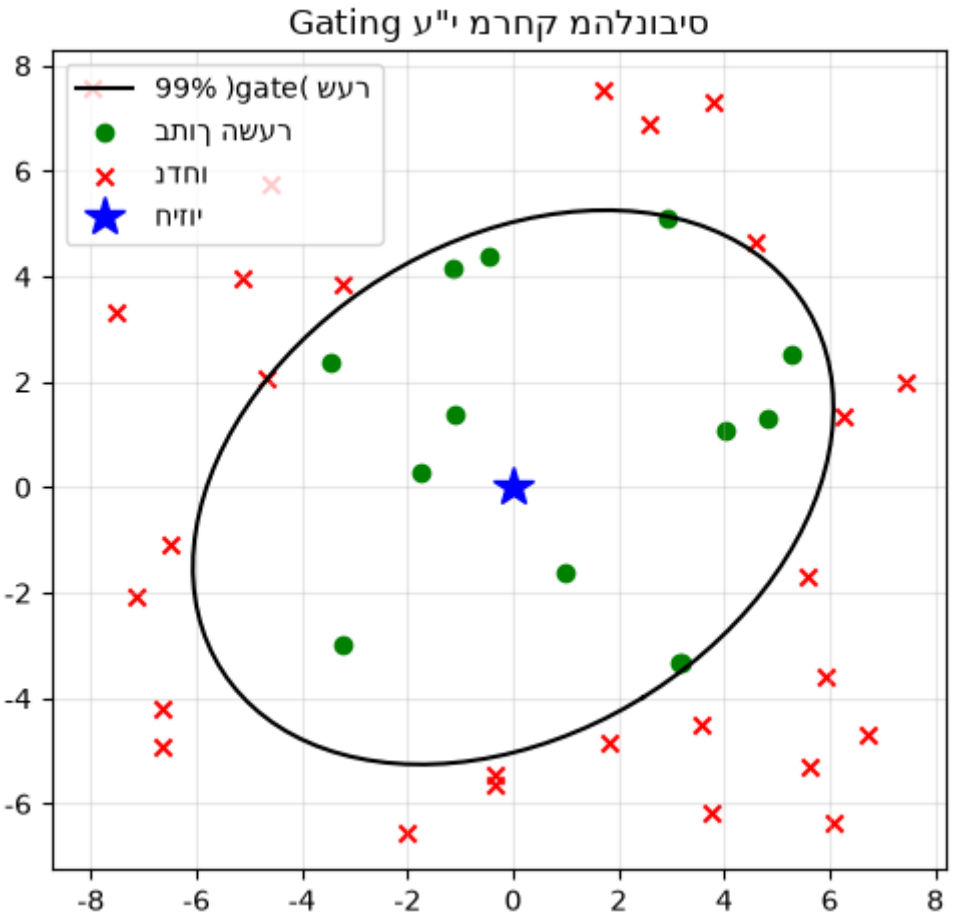
d^2

מתפלג

χ^2

עם דרגות חופש כמספר ממדי המדידה.

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import permutations
rng = np.random.default_rng(3)

def maha2(z, zhat, S):
    d = (z - zhat).reshape(-1,1)
    return float((d.T @ np.linalg.inv(S) @ d).item())

# חידוי מטרה במישור עם שונות innovation S; שער chi2 2 עם dof, 99% -> 9.21
zhat = np.array([0.,0.]); S = np.array([[4.,1.],[1.,3.]]); gamma = 9.21
cands = rng.uniform(-8,8,size=(40,2))
ingate = np.array([maha2(z,zhat,S) <= gamma for z in cands])
th=np.linspace(0,2*np.pi,100); v,Q=np.linalg.eigh(S)
ell=(Q@np.diag(np.sqrt(gamma*v))@np.array([np.cos(th),np.sin(th)]))
plt.figure(figsize=(5,5))
plt.plot(ell[0],ell[1],'k',label='שער (gate) 99%')
plt.scatter(*cands[ingate].T,c='g',label='בתוך השער')
plt.scatter(*cands[~ingate].T,c='r',marker='x',label='נדחו')
plt.plot(*zhat,'b*',ms=15,label='חידוי'); plt.gca().set_aspect('equal')
plt.legend(); plt.grid(alpha=.3); plt.title('Gating מרחק מהלנוביס ע"י'); plt.tight_layout();
plt.show()
print(f'{ingate.sum()} מתוך {len(cands)} מדידות נכנסו לשער')
```



מתוך 40 מדידות נכנסו לשער 13

ב. שיוך 'השכן הקרוב הגלובלי' (GNN — Global Nearest Neighbor)¶

כאשר יש כמה מטרות וכמה מדידות, GNN פותר **בעיית שיבוץ (assignment)**: למצוא את ההתאמה חד-חד-ערכית בין מסלולים למדידות שממזערת את עלות השיוך הכוללת (סכום מרחקי מהלנוביס):

$$\min_{\text{שיבוץ}} \sum_{(i,j)} d_{ij}^2$$

פתרון אופטימלי: אלגוריתם הונגרי / auction. כאן נדגים בכוח גס (brute force) עבור מספר קטן של מטרות.

```
def gnn_assign(cost):
    """שיבוץ אופטימלי (מינימום עלות) בכוח גס — למטריצות קטנות."""
    n,m = cost.shape; best=None; best_c=np.inf
    rows=range(n)
    for perm in permutations(range(m), n):
        c=sum(cost[i,perm[i]] for i in rows)
        if c<best_c: best_c=c; best=perm
    return list(zip(rows,best)), best_c
```

```
# 3 מדידות 3-ו (חיזויים) מסלולים
tracks = np.array([[0.,0.],[10.,0.],[5.,8.]])
meas   = np.array([[0.5,0.4],[9.4,0.7],[5.3,7.6]])
Sg = np.eye(2)*1.0
cost = np.array([[maha2(m,t,Sg) for m in meas] for t in tracks])
pairs, total = gnn_assign(cost)
print('(מדידה=עמודה ,מסלול=שורה) עלויות:'); print(np.round(cost,1))
print('שיבוץ GNN:', [(f'מסלול{i}',f'מדידה{j}') for i,j in pairs], f'  עלות={total:.2f}')
```

```
(מדידה=עמודה ,מסלול=שורה) עלויות:
[[ 0.4 88.9 85.8]
 [90.4  0.8 79.8]
 [78.  72.6  0.3]]
שיבוץ GNN: [('מסלול0' ,'מדידה0'), ('מסלול1' ,'מדידה1'), ('מסלול2' ,'מדידה2')]   1.51=ות'
```

ג. PDA — Probabilistic Data Association (מטרה יחידה ב-clutter)¶

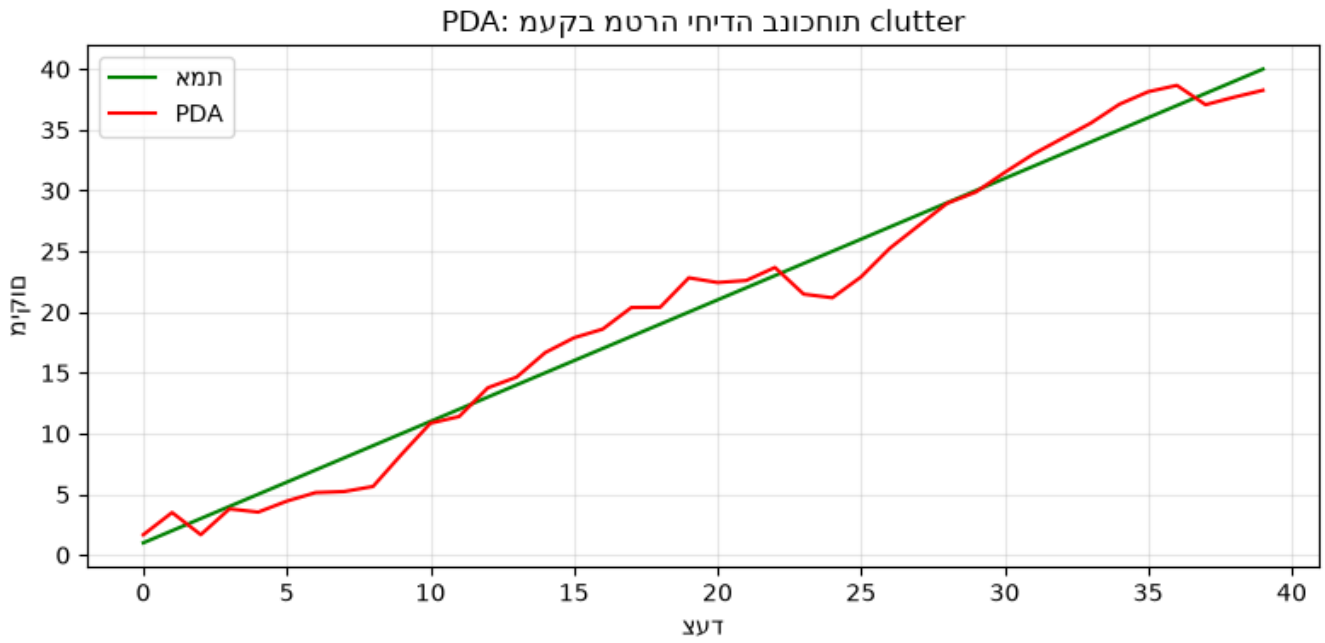
במקום לבחור מדידה אחת, **PDA משקלל את כל המדידות בשער** לפי הסתברות השיוך שלהן. עבור מדידות z_i בשער, הסתברות שמדידה i היא הנכונה:

$$\beta_i = \frac{\mathcal{L}_i}{1 - P_D P_G + \sum_k \mathcal{L}_k}, \qquad \mathcal{L}_i = \frac{P_D}{\lambda}\,\mathcal{N}(z_i;\hat{z},S)$$

-1 β_0 (אף מדידה אינה נכונה). העדכון משתמש ב**חדשה משוקללת** $\bar{y}=\sum_i \beta_i y_i$, עם תוספת לשונות ('spread of innovations') שמבטאת את אי-ודאות השיוך.

```
def pda_update(x, P, F, H, Q, R, zs, PD=0.9, PG=0.99, lam=2e-4):
    x=F@x; P=F@P@F.T+Q                       # predict
    zhat=H@x; S=H@P@H.T+R; K=P@H.T@np.linalg.inv(S)
    if len(zs)==0:
        return x, P, None
    ys=[z-zhat for z in zs]
    norm=1.0/np.sqrt(np.linalg.det(2*np.pi*S))
    Ls=np.array([PD/lam*norm*np.exp(-0.5*float(y@np.linalg.inv(S)@y)) for y in ys])
    b0=1-PD*PG; denom=b0+Ls.sum()
    betas=Ls/denom; beta0=b0/denom
    ybar=sum(b*y for b,y in zip(betas,ys))
    x=x+K@ybar
    # spread of innovations term
    spread=sum(b*np.outer(y,y) for b,y in zip(betas,ys))-np.outer(ybar,ybar)
    Pc=(np.eye(len(x))-K@H)@P
```

```
    P=beta0*P+(1-beta0)*Pc+K@spread@K.T
    return x, P, betas

# 1 מטרה D מדידות מיקום ב ,נעה-clutter
T=1.0; F=np.array([[1,T],[0,1.]]); H=np.array([[1.,0]]); Q=np.array([[.05,.1],[.1,.2]]);
R=np.array([[4.]])
x=np.array([0.,1.]); P=np.diag([4.,4.]); true=x.copy()
est=[]; tru=[]
for k in range(40):
    true=F@true; tru.append(true[0])
    zs=[np.array([true[0]+2*rng.standard_normal()])]          # מדידה אמיתית
    zs+=[np.array([true[0]+rng.uniform(-15,15)]) for _ in range(rng.poisson(2))]  # clutter
    x,P,_=pda_update(x,P,F,H,Q,R,zs)
    est.append(x[0])
plt.figure(figsize=(8,4))
plt.plot(tru,'g',label='אמת'); plt.plot(est,'r',label='PDA')
plt.title('PDA: מעקב מטרה יחידה בנוכחות clutter'); plt.legend(); plt.grid(alpha=.3)
plt.xlabel('צעד'); plt.ylabel('מיקום'); plt.tight_layout(); plt.show()
print(f'RMSE = {np.sqrt(np.mean((np.array(est)-np.array(tru))**2)):.2f}')
```




```
RMSE = 1.73
```

ד. הרחבות: JPDA ו-MHT¶

- **JPDA** (Joint PDA) — מרחיב PDA למספר מטרות, ומחשב הסתברויות שיוך משותפות תוך מניעת התנגשויות.
- **MHT** (Multiple Hypothesis Tracking) — שומר **עץ השערות** של שיוכים לאורך זמן ומכריע בדיעבד; עוצמתי בסביבות צפופות אך יקר חישובית (נדרש גיזום/pruning).
- גישות מודרניות (כמו של Meyer): **belief propagation** על גרף פקטורים לשיוך נתונים סקיילבילי.

הקשר ל-fusion: שיוך נכון הוא תנאי מקדים לכל מסנן (מחברת 03) — 'שיוך שגוי' הוא מקור השגיאה הדומיננטי במעקב רב-מטרתי, חשוב יותר מבחירת המסנן עצמו.

סיכום¶

- **Gating** מצמצם מועמדים דרך מרחק מהלנוביס (χ^2).
- **GNN** = בעיית שיבוץ אופטימלית (החלטה קשיחה).
- **PDA/JPDA** = שקלול הסתברותי רך של כל המדידות בשער; חסין יותר ל-clutter.
- **MHT** דוחה הכרעה ושומר השערות מרובות.

06 · מעקב מטרות מבוזר (Distributed Target Tracking)¶

מלווה את ההרצאה: Felix Govaers, FUSION Boot Camp 2026 — Distributed Target Tracking.

שקופיות הרצאה זו **אינן** ב-PDF. המחברת מבוססת על הספרות הקנונית של היתוך מבוזר.

במעקב **מבוזר**, מספר צמתים/חיישנים מריצים מסננים מקומיים ומחליפים מסלולים (tracks) ולא מדידות גולמיות. האתגר המרכזי: **קורלציה נסתרת** בין המסלולים (רעש תהליך משותף, מידע משותף) — מיזוג נאיבי נעשה אופטימי-יתר (over-confident).

א. מסנן המידע (Information Filter)¶

צורת המידע של קלמן מחליפה

$$(\hat{x}, P)$$

ב**וקטור מידע**

$$y = P^{-1}\hat{x}$$

ו**מטריצת מידע**

$$Y = P^{-1}$$

. יתרונה: **עדכון מדידה הופך לחיבור** — אידיאלי להיתוך מבוזר:

$$Y_{k|k} = Y_{k|k-1} + H^T R^{-1} H, \qquad y_{k|k} = y_{k|k-1} + H^T R^{-1} z$$

מספר חיישנים בלתי-תלויים פשוט **מחברים** את תרומות המידע שלהם — זוהי הכללה ישירה של משוואת ה-Fusion ממחברת 01 (

$$Y_F = Y_1 + Y_2$$

).

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(5)

def fuse_information(estimates):
    """היתוך אופטימלי של אומדנים *בלתי-תלויים* בצורת מידע."""
    Y = sum(np.linalg.inv(P) for _,P in estimates)
    y = sum(np.linalg.inv(P)@x for x,P in estimates)
    P = np.linalg.inv(Y); return P@y, P

# 2 שלושה חיישנים מודדים מיקום 2D, שונויות שונות, של אותה מטרה
true = np.array([3.,2.])
Ps = [np.array([[5,1.],[1,2]]), np.array([[2,-.5],[-.5,4]]), np.array([[3,0.],[0,1]])]
ests = [(true + np.linalg.cholesky(P)@rng.standard_normal(2), P) for P in Ps]
xf, Pf = fuse_information(ests)
print('אומד ממוזג:', np.round(xf,2), ' det(P_fused)=%.3f' % np.linalg.det(Pf))
for i,(_,P) in enumerate(ests): print(f'  det(P_{i})={np.linalg.det(P):.3f}')
print('-> המידע הממוזג (det קטן) טוב מכל חיישן בודד.')
```

```
[1.64 3.33] :אומד ממוזג  det(P_fused)=0.515
  det(P_0)=9.000
  det(P_1)=7.750
  det(P_2)=3.000
-> המידע הממוזג (det קטן) טוב מכל חיישן בודד.
```

ב. הבעיה: קורלציה נסתרת והיתוך נאיבי¶

מסלולים מצמתים שונים **אינם בלתי-תלויים** — הם חולקים רעש תהליך של אותה מטרה, ולעיתים מידע עבר משותף (rumor propagation / double counting). היתוך 'מידע' נאיבי מתעלם מכך ומפיק שונות **קטנה מדי** (over-confident), כלומר מסנן **לא עקבי**.

```
# סימולציה: שני מסלולים עם רעש תהליך משותף (קורלציה אמיתית)
mc=3000; common=0.8   # שיעור השונות המשותפת
naive_in, ci_in = 0, 0
for _ in range(mc):
    e_common = np.sqrt(common)*rng.standard_normal(2)
    P1=P2=np.eye(2)
    x1=true+e_common+np.sqrt(1-common)*rng.standard_normal(2)
    x2=true+e_common+np.sqrt(1-common)*rng.standard_normal(2)
    # היתוך נאיבי (מניח אי-תלות)
    xf,Pf=fuse_information([(x1,P1),(x2,P2)])
    d=(xf-true); nees=float(d@np.linalg.inv(Pf)@d)
    naive_in += nees<=5.99   # chi2_2 95%
naive_cov = naive_in/mc
print(f'כיסוי אזור-ביטחון של היתוך נאיבי: {naive_cov:.1%} (מתחת לכך — 95%~ אמור להיות = over-confident)')
```

```
over-confid = אמור להיות ~95% — מתחת לכך) כיסוי אזור-ביטחון של 95% של היתוך נאיבי: 81.0%
```

ג. Covariance Intersection (CI)¶

כש**הקורלציה לא ידועה**, CI מבטיח עקביות ללא הנחת אי-תלות. הוא משלב בצורת מידע עם משקל $\omega \in [0,1]$:

$$P_{CI}^{-1} = \omega P_1^{-1} + (1-\omega) P_2^{-1}, \qquad P_{CI}^{-1}\hat{x}_{CI} = \omega P_1^{-1}\hat{x}_1 + (1-\omega) P_2^{-1}\hat{x}_2$$

ω

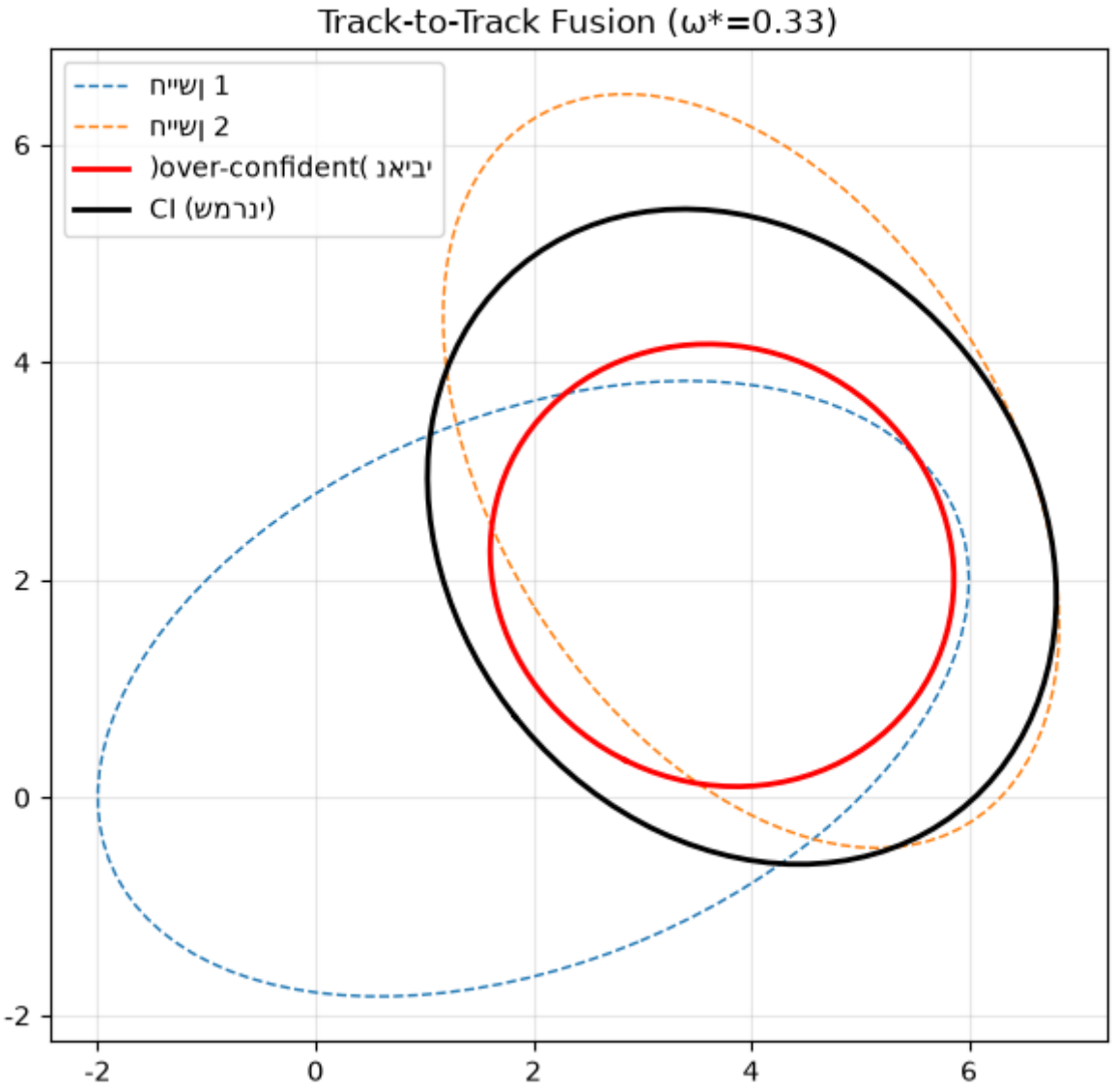
נבחר בד"כ למזעור $\det(P_{CI})$. התוצאה **שמרנית** (לא over-confident) לכל קורלציה.

```
def covariance_intersection(x1,P1,x2,P2):
    I1,I2=np.linalg.inv(P1),np.linalg.inv(P2); best=None
    for w in np.linspace(0.01,0.99,99):
        Pi=np.linalg.inv(w*I1+(1-w)*I2); d=np.linalg.det(Pi)
        if best is None or d<best[0]: best=(d,w,Pi)
    _,w,Pci=best
    xci=Pci@(w*I1@x1+(1-w)*I2@x2)
    return xci,Pci,w
```

```
ci_in=0
for _ in range(mc):
    e_common=np.sqrt(common)*rng.standard_normal(2)
    x1=true+e_common+np.sqrt(1-common)*rng.standard_normal(2)
    x2=true+e_common+np.sqrt(1-common)*rng.standard_normal(2)
    xci,Pci,_=covariance_intersection(x1,np.eye(2),x2,np.eye(2))
    d=(xci-true); nees=float(d@np.linalg.inv(Pci)@d)
    ci_in += nees<=5.99
print(f'כיסוי נאיבי: {naive_cov:.1%}   |   כיסוי CI: {ci_in/mc:.1%}  (CI שמרני/עקבי)')
```

```
כיסוי   |   81.0% :כיסוי נאיבי CI: 95.2%  (CI שמרני/עקבי)
```

```
# שמרני CI, השוואה ויזואלית של אליפסות: נאיבי קטן מדי
x1=np.array([2.,1.]); x2=np.array([4.,3.]); P1=np.array([[4,1.],[1,2]]); P2=np.array([[2,-1.],
[-1,3]])
xn,Pn=fuse_information([(x1,P1),(x2,P2)]); xci,Pci,w=covariance_intersection(x1,P1,x2,P2)
def ell(P,m,**k):
    th=np.linspace(0,2*np.pi,100); v,Q=np.linalg.eigh(P)
    e=Q@np.diag(2*np.sqrt(v))@np.array([np.cos(th),np.sin(th)]); return e[0]+m[0],e[1]+m[1]
plt.figure(figsize=(6,6))
for P,m,st,lab in [(P1,x1,'C0--','1 חיישן'),(P2,x2,'C1--','2 חיישן'),
                   (Pn,xn,'r-','נאיבי (over-confident)'),(Pci,xci,'k-','CI (שמרני)')]:
    ex,ey=ell(P,m); plt.plot(ex,ey,st,lw=2 if st in('r-','k-') else 1,label=lab)
plt.gca().set_aspect('equal'); plt.legend(); plt.grid(alpha=.3)
plt.title(f'Track-to-Track Fusion (ω*={w:.2f})'); plt.tight_layout(); plt.show()
```



סיכום¶

- **מסנן המידע** הופך היתוך לחיבור — בסיס להיתוך מבוזר; חיישנים בלתי-תלויים:
$$Y_F = \sum Y_i$$
.
- **קורלציה נסתרת** בין מסלולים גורמת להיתוך נאיבי להיות over-confident (לא עקבי).
- **Covariance Intersection** מבטיח עקביות לכל קורלציה לא-ידועה — מחיר: אומד שמרני יותר.
- נושאים נוספים בהרצאה: distributed/decentralized Kalman, accumulated state densities, רשתות חיישנים.

07 · היתוך ושיוך עם תכונות / סיווג (Attributes & Classification)¶

מלווה את ההרצאה: Yaakov Bar-Shalom — Fusion and Association with Attributes/Classification.

שקופיות הרצאה זו **אינן** ב-PDF. המחברת מבוססת על הספרות הקנונית ועל התייחסות Blasch במבוא ל-'Object level — Bayes versus Dempster-Shafer'.

עד כה עסקנו במצב **קינמטי** (מיקום/מהירות). תכונות (attributes) — כמו **מחלקת המטרה** (class/ID) — מוסיפות מימד נוסף ל-fusion: הן משפרות גם את ה**שיוך** (אסור לשייך מדידת 'משאית' למסלול 'מטוס') וגם את ה**זיהוי**.

א. סיווג בייסיאני וצבירה רצופה (Sequential Bayesian Fusion)¶

נתון prior על המחלקות

$P(c)$

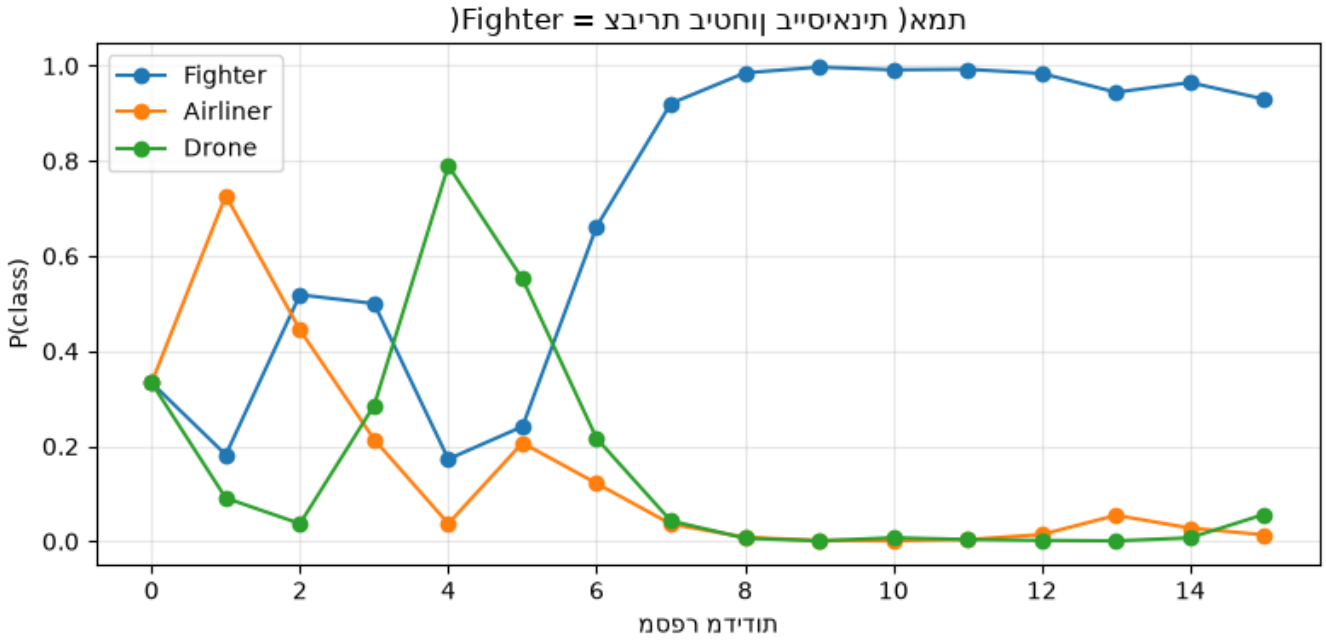
ו'מטריצת בלבול' (confusion matrix)

$P(z|c)$

של החיישן. כל מדידה מעדכנת את ההסתברות הפוסטריורית בכלל בייס, וכך **צוברים ביטחון** על פני זמן/חיישנים:

$$P(c|z_{1:k}) \propto P(z_k|c)\,P(c|z_{1:k-1})$$

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(9)
classes = ['Fighter','Airliner','Drone']
# מטריצת בלבול: שורה=מחלקה אמיתית, עמודה=דיווח החיישן
C = np.array([[0.70,0.20,0.10],
              [0.15,0.80,0.05],
              [0.10,0.10,0.80]])

def bayes_fuse(prior, reports, C):
    p = prior.copy(); hist=[p.copy()]
    for r in reports:
        p = C[:,r]*p; p/=p.sum(); hist.append(p.copy())
    return p, np.array(hist)

true_c = 0  # Fighter
reports = [np.argmax(rng.multinomial(1, C[true_c])) for _ in range(15)]
post, hist = bayes_fuse(np.ones(3)/3, reports, C)
plt.figure(figsize=(8,4))
for j,name in enumerate(classes): plt.plot(hist[:,j],'o-',label=name)
plt.title(f'צבירת ביטחון בייסיאנית (אמת = {classes[true_c]})')
plt.xlabel('מספר מדידות'); plt.ylabel('P(class)'); plt.legend(); plt.grid(alpha=.3)
plt.tight_layout(); plt.show()
print('פוסטריור סופי:', dict(zip(classes, np.round(post,3))))
```




```
פוסטריור סופי: {'Fighter': np.float64(0.93), 'Airliner': np.float64(0.013), 'Drone': np
```

ב. תורת ההחלטה: מטריצת בלבול ועלות סיכון¶

ההחלטה אינה תמיד 'המחלקה הסבירה ביותר' — אפשר לשקלל **עלויות שגיאה** (למשל סיווג שגוי של מטוס נוסעים כמטרה עוין). ההחלטה הבייסיאנית האופטימלית ממזערת את הסיכון

$$R(a|z) = \sum_c L(a,c)\,P(c|z)$$

.

```
# מטריצת בלבול אמפירית (5 מדידות אחרי MAP) — הערכת ביצועי המסווג ע"י סימולציית החלטות
K=5; trials=4000; conf=np.zeros((3,3))
for _ in range(trials):
    tc=rng.integers(3)
    rep=[np.argmax(rng.multinomial(1,C[tc])) for _ in range(K)]
    post,_=bayes_fuse(np.ones(3)/3, rep, C)
    conf[tc, np.argmax(post)]+=1
conf=conf/conf.sum(1,keepdims=True)
print('מטריצת בלבול אחרי %d מדידות: (שורה=אמת, עמודה=החלטה)'%K)
print('          '+'  '.join(f'{c:>9}' for c in classes))
for i,c in enumerate(classes): print(f'{c:>9} '+'  '.join(f'{v:9.2f}' for v in conf[i]))
print(f'דיוק כולל = {np.trace(conf)/3:.1%}  (לעומת ~{np.trace(C)/3:.1%} ממדידה בודדת)')
```

```
מטריצת בלבול אחרי 5 מדידות: (שורה=אמת, עמודה=החלטה)
           Fighter   Airliner      Drone
  Fighter      0.93       0.06       0.02
 Airliner      0.06       0.93       0.01
    Drone      0.02       0.01       0.97
דיוק כולל = 94.2%  (לעומת ~76.7% ממדידה בודדת)
```

ג. Dempster–Shafer מול Bayes¶

תורת ה**עדויות (Dempster-Shafer)** מכלילה את בייס ומאפשרת לייצג **בורות (ignorance)** ע"י הקצאת מסה לקבוצות מחלקות (לא רק ליחידים). כלל השילוב של דמפסטר:

$$m_{12}(A) = \frac{1}{1-K} \sum_{B \cap C = A} m_1(B)\, m_2(C), \quad K = \sum_{B \cap C = \emptyset} m_1(B)\, m_2(C)$$

כאשר

K

הוא ה**קונפליקט**. כשהקונפליקט גבוה, נורמליזציית דמפסטר עלולה לתת תוצאות לא-אינטואיטיביות (פרדוקס Zadeh).

```
from itertools import combinations
# מסגרת הבחנה: {F,A,D}; מסות מוקצות לתת-קבוצות (frozenset)
frame=['F','A','D']
subsets=[frozenset(s) for r in range(1,4) for s in combinations(frame,r)]
def ds_combine(m1,m2):
    out={}; K=0.0
    for B,mb in m1.items():
        for Cc,mc in m2.items():
            inter=B&Cc
            if inter: out[inter]=out.get(inter,0)+mb*mc
            else: K+=mb*mc
    return {k:v/(1-K) for k,v in out.items()}, K

# חיישן 1 בטוח יחסית ב-Fighter אך משאיר בורות; חיישן 2 תומך ב-Fighter/Drone
m1={frozenset(['F']):0.6, frozenset(['F','A']):0.2, frozenset(frame):0.2}
m2={frozenset(['F']):0.5, frozenset(['D']):0.3, frozenset(frame):0.2}
comb,K=ds_combine(m1,m2)
print(f'קונפליקט K = {K:.3f}')
for s,v in sorted(comb.items(), key=lambda kv:-kv[1]):
    print(f'  m({set(s)}) = {v:.3f}')
# Belief ו-Plausibility ל-{F}
A=frozenset(['F'])
bel=sum(v for s,v in comb.items() if s<=A)
pl =sum(v for s,v in comb.items() if s&A)
print(f'Bel(Fighter)={bel:.3f}, Pl(Fighter)={pl:.3f}  (מרווח אי-הוודאות)')
```

```
קונפליקט K = 0.240
  m({'F'}) = 0.816
  m({'D'}) = 0.079
  m({'F', 'A'}) = 0.053
  m({'D', 'F', 'A'}) = 0.053
Bel(Fighter)=0.816, Pl(Fighter)=0.921  (מרווח אי-הוודאות)
```

סיכום¶

- תכונות/מחלקה משפרות גם **שיוך** וגם **זיהוי**; היתוך בייסיאני **צובר ביטחון** על פני מדידות.
- **תורת ההחלטה** מאפשרת לשקלל עלויות שגיאה אסימטריות.
- **Dempster-Shafer** מייצג בורות במפורש; להיזהר מקונפליקט גבוה.

- בפועל, fusion מודרני משלב סיווג קינמטי+תכונתי (joint tracking & classification) — חיבור בין מחברות 03–04 לכאן.

08 · היסק מבוזר והיתוך מידע (Distributed Inference)¶

מלווה את ההרצאה: Pramod Varshney — Distributed Inference and Information Fusion.

שקופיות הרצאה זו **אינן** ב-PDF. המחברת מבוססת על הספרות הקנונית של גילוי מבוזר (Varshney, Distributed Detection and Data Fusion, 1997).

בגילוי **מבוזר**, כל חיישן מקבל החלטה מקומית (ביט) ושולח אותה למרכז היתוך (fusion center) במקום את הנתון הגולמי — חוסך רוחב פס. השאלות: איזה **כלל היתוך** למרכז? ואיך נקבעים ה**ספים** המקומיים?

א. עקומת ROC ופשרת גילוי/אזעקת-שווא¶

כל חיישן מאופיין ב-

(P_D, P_{FA})

. עבור גילוי אות

A

ברעש גאוסיאני

σ

עם סף

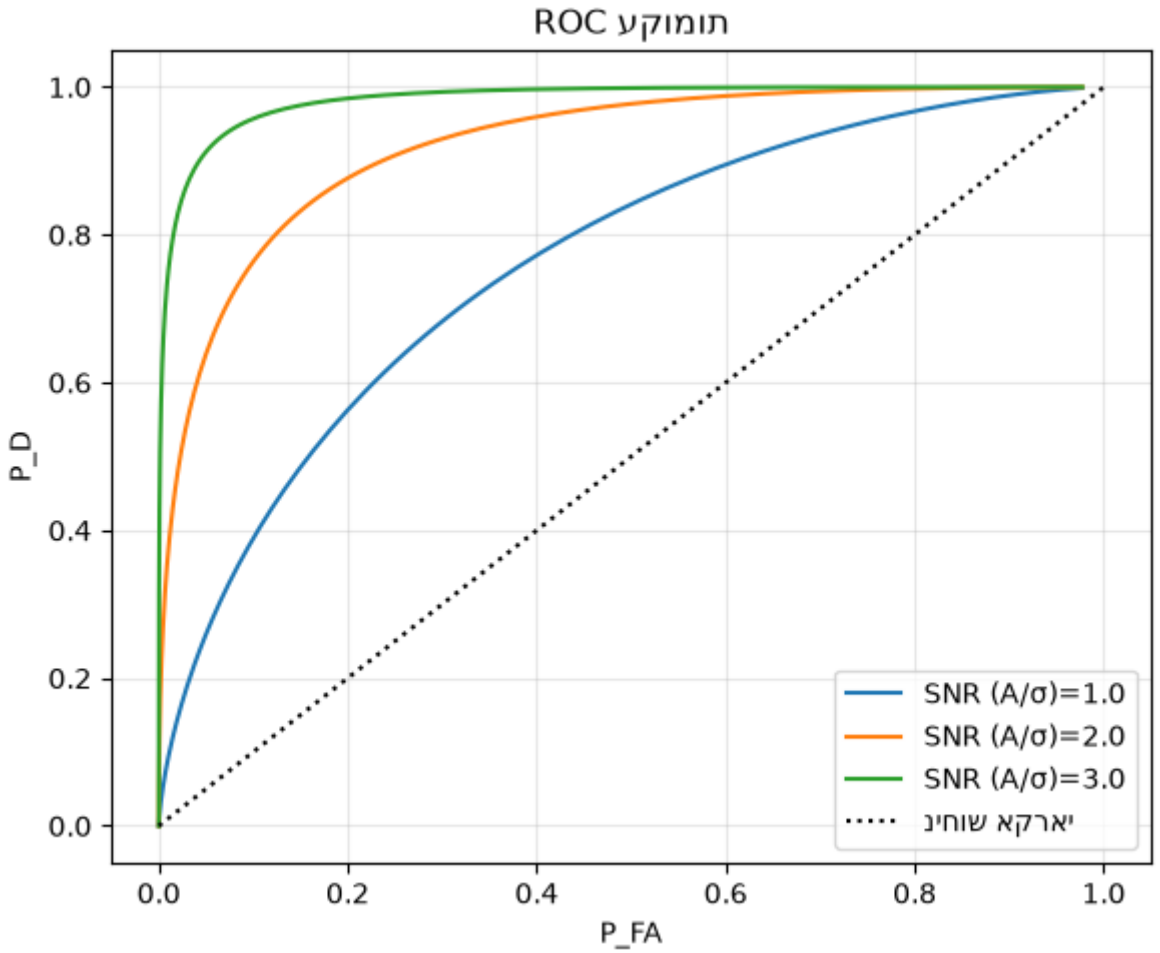
τ

:

$$P_{FA} = Q(\tau/\sigma), \qquad P_D = Q((\tau - A)/\sigma), \qquad Q(x) = \tfrac{1}{2}\mathrm{erfc}(x/\sqrt{2})$$

ה-SNR קובע את עקומת ה-ROC — ככל שהיא 'גבוהה' יותר (קרובה לפינה השמאלית-עליונה), הגילוי טוב יותר.

```
import numpy as np
import matplotlib.pyplot as plt
from math import erfc, sqrt
rng = np.random.default_rng(2)
Q = lambda x: 0.5*erfc(x/sqrt(2))
Qv = np.vectorize(Q)

taus = np.linspace(-2,8,300)
plt.figure(figsize=(6,5))
for A in [1.0, 2.0, 3.0]:
    pfa=Qv(taus); pd=Qv(taus-A)
    plt.plot(pfa,pd,label=f'SNR (A/σ)={A}')
plt.plot([0,1],[0,1],'k:',label='ניחוש אקראי')
plt.xlabel('P_FA'); plt.ylabel('P_D'); plt.title('עקומות ROC'); plt.legend(); plt.grid(alpha=.
3)
plt.tight_layout(); plt.show()
```



ב. כללי היתוך החלטות: OR / AND / רוב (k-out-of-N)¶

עם

N

חיישנים בלתי-תלויים מותנית, מרכז ההיתוך מחליט H1 אם לפחות

k

חיישנים דיווחו '1':

- **OR** (

$k = 1$

) — רגיש מאוד,

P_D

גבוה אך גם

P_{FA}

גבוה.
- **AND** (

$k = N$

) — שמרני,

P_{FA}

נמוך אך

P_D

נמוך.

- **רוב (**

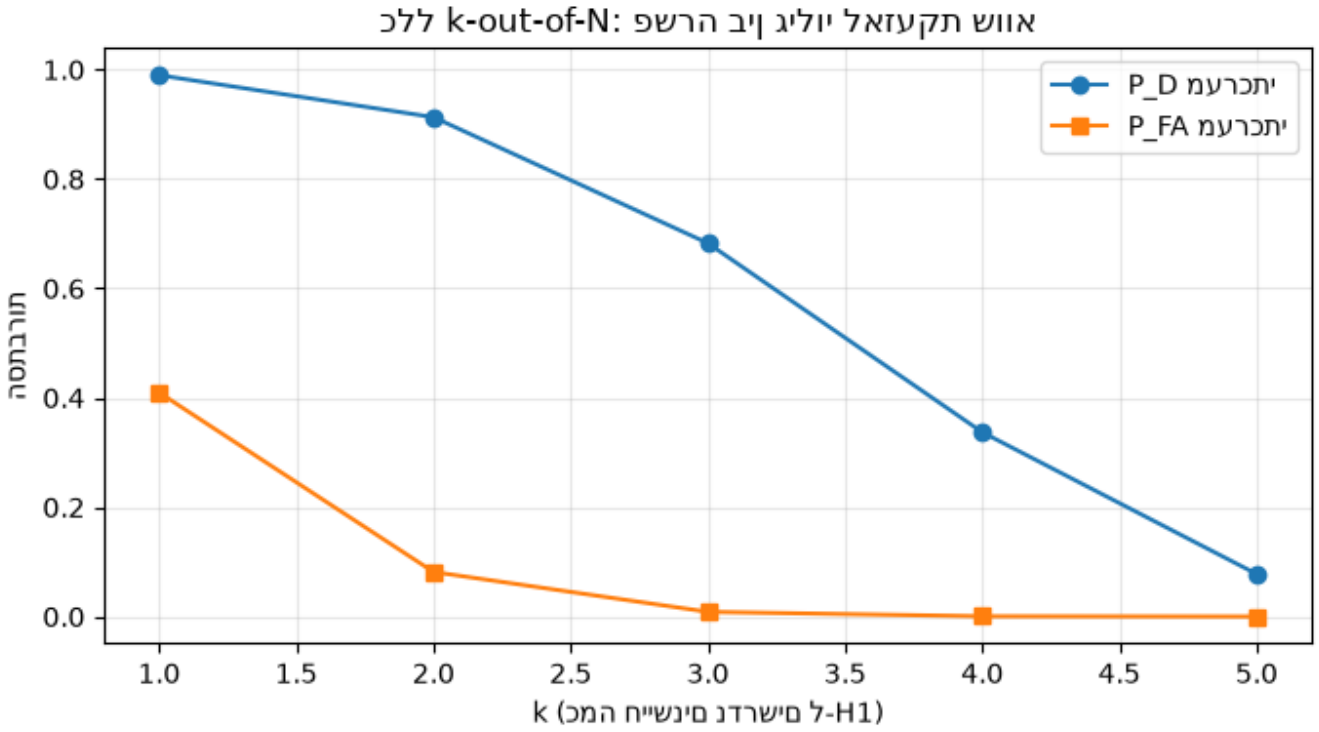
$k = \lceil N/2 \rceil$

) — איזון.

$$P_D^{fus} = \sum_{i=k}^{N} \binom{N}{i} P_D^i (1 - P_D)^{N-i}$$

```
from math import comb
def kN(p, N, k): return sum(comb(N,i)*p**i*(1-p)**(N-i) for i in range(k,N+1))
N=5; pd, pfa = 0.6, 0.1
print(f'חיישן בודד: P_D={pd}, P_FA={pfa}')
for name,k in [('OR (1/5)',1),('(3/5) רוב',3),('AND (5/5)',5)]:
    print(f'{name:12s}  P_D={kN(pd,N,k):.3f}  P_FA={kN(pfa,N,k):.4f}')

ks=range(1,N+1)
plt.figure(figsize=(7,4))
plt.plot(list(ks),[kN(pd,N,k) for k in ks],'o-',label='P_D מערכתי')
plt.plot(list(ks),[kN(pfa,N,k) for k in ks],'s-',label='P_FA מערכתי')
plt.xlabel('k (כמה חיישנים נדרשים ל-H1)'); plt.ylabel('הסתברות')
plt.title('כלל k-out-of-N: פשרה בין גילוי לאזעקת שווא'); plt.legend(); plt.grid(alpha=.3)
plt.tight_layout(); plt.show()
```

```
חיישן בודד: P_D=0.6, P_FA=0.1
OR (1/5)      P_D=0.990  P_FA=0.4095
(3/5) רוב     P_D=0.683  P_FA=0.0086
AND (5/5)     P_D=0.078  P_FA=0.0000
```



ג. כלל ההיתוך האופטימלי של Chair–Varshney¶

כשידועים

$(P_{D,i}, P_{FA,i})$

של כל חיישן, כלל ה-LRT האופטימלי משקלל כל החלטה מקומית

$u_i \in \{0,1\}$

לפי **אמינותה**:

$$\sum_i [u_i \ln \frac{P_{D,i}}{P_{FA,i}} + (1-u_i) \ln \frac{1-P_{D,i}}{1-P_{FA,i}}] \gtrless \ln \frac{P_0}{P_1}$$

חיישנים אמינים יותר מקבלים משקל גדול יותר — בניגוד להצבעת רוב 'שווה'.

```
def chair_varshney(u, PD, PFA, P1=0.5):
    P0=1-P1; s=0.0
    for ui,pd,pf in zip(u,PD,PFA):
        s += np.log(pd/pf) if ui==1 else np.log((1-pd)/(1-pf))
    return 1 if s >= np.log(P0/P1) else 0

# 5 חיישנים הטרוגניים: חלקם אמינים, חלקם רועשים
PD =np.array([0.95,0.9,0.6,0.55,0.7])
PFA=np.array([0.02,0.05,0.3,0.4,0.2])
def majority(u): return 1 if sum(u)>=3 else 0

trials=20000; H1prob=0.5; cv_ok=maj_ok=0
for _ in range(trials):
    H=int(rng.random()<H1prob)
    u=[int(rng.random()<(PD[i] if H else PFA[i])) for i in range(5)]
    cv_ok += chair_varshney(u,PD,PFA)==H
    maj_ok+= majority(u)==H
print(f'דיוק הצבעת רוב     : {maj_ok/trials:.1%}')
print(f'דיוק Chair-Varshney: {cv_ok/trials:.1%}  (משקלל אמינות <- טוב יותר עם חיישנים
הטרוגניים)')
```

```
דיוק הצבעת רוב     :  93.5%
דיוק Chair-Varshney: 97.8%  (משקלל אמינות <- טוב יותר עם חיישנים הטרוגניים)
```

ד. הקשר רחב: consensus וקונפידנס¶

- **Consensus / gossip** — צמתים ברשת ללא מרכז מתכנסים לאומד גלובלי דרך החלפות מקומיות בלבד.
- **קוונטיזציה** — שליחת ביטים ספורים במקום ערכים מלאים; פשרת רוחב-פס מול דיוק.
- **תקשורת לא-אמינה** — אובדן/השהיית הודעות מחייבים היתוך חסין. כל אלה הם הכללות מבוזרות של אותו עיקרון: **צבירת מידע מקטינה אי-ודאות** (משוואת ה-Fusion, מחברת 01).

סיכום¶

- **ROC** מאפיין כל חיישן; ה-SNR קובע את גבול הביצועים.
- כללי **k-out-of-N** (AND/רוב/OR) נותנים פשרות שונות בין

P_D

ל-

P_{FA}

.

- **Chair–Varshney** הוא כלל ההיתוך האופטימלי — משקלל לפי אמינות, עדיף על הצבעת רוב כשהחיישנים הטרוגניים.

09 · היתוך ברמה גבוהה (High-level Fusion)¶

מלווה את ההרצאה: Erik Blasch, FUSION Boot Camp 2026 — High-level Fusion (הרצאה 7).

שקופיות הרצאה 7 **אינן** ב-PDF, אך הרצאת המבוא של Blasch (מחברת 01) מכסה את היסודות. המחברת מבוססת עליהם ועל הספרות (Blasch, Bossé, Lambert, High-Level Information Fusion).

רמות **L0/L1** (מחברות 02–05) עוסקות ב**עצמים**: היכן ומה. רמות **L2–L5** עוסקות ב**משמעות**: מה המצב (situation), מה האיום (threat/impact), כיצד לכוון חיישנים (process refinement), ומה האדם צריך (user refinement).

א. תזכורת — רמות ה-JDL/DFIG הגבוהות¶

רמה	שם	שאלה	טכניקות
L2	Situation Assessment	מה היחסים בין הישויות?	Bayesian nets, גרפים, ontologies
L3	Threat / Impact	מה ההשלכות והכוונה (intent)?	game theory, הסקה אבדוקטיבית
L4	Process Refinement	איך לכוון את החיישנים?	sensor management (מחברת 08)
L5	User Refinement	מה האדם צריך לראות/להחליט?	HMI, ויזואליזציה, הסבר

מאפיין מבדיל: ב-L2+ ההסקה היא **אבדוקטיבית** (ההסבר הסביר ביותר) ולעיתים **soft** (טקסט/דיווחי אדם), בניגוד להסקה הכמותית/hard ברמות הנמוכות.

ב. הערכת איום (Threat/Impact Assessment) — דוגמה מספרית¶

נדגים היתוך **קינמטי + זהותי** לדירוג איום. לכל מטרה נחשב מדד איום שמשלב:

- **CPA** (Closest Point of Approach) — כמה קרוב המסלול יעבור לנכס המוגן (קינמטי, מ-L1).
- **TTI** (Time-To-Intercept) — מתי (דחיפות).
- **P(hostile)** — הסתברות עוינות מהסיווג (מחברת 07). מדד האיום:

$$\text{threat} = P(\text{hostile}) \cdot f_{CPA} \cdot f_{TTI}$$

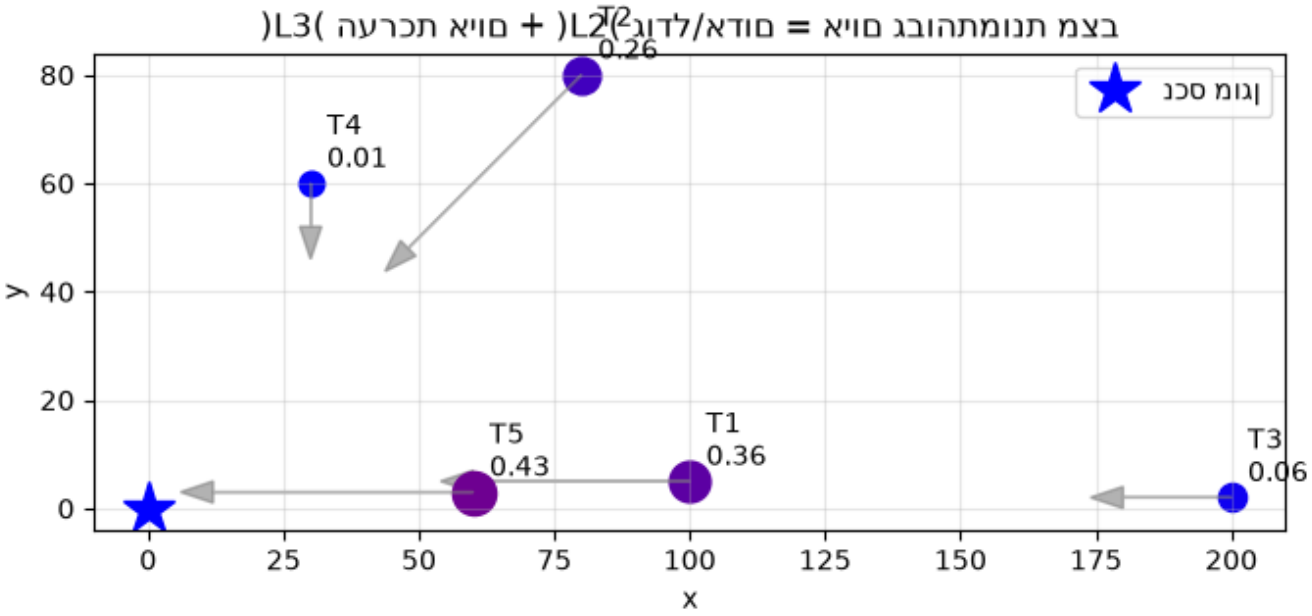
.

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(4)
asset = np.array([0.,0.])    # הנכס המוגן בראשית

def cpa_tti(pos, vel):
    """Closest Point of Approach (מרחק) יחסית לנכס בראשית, וזמן אליו."""
    r=pos-asset; v=vel
    if v@v < 1e-9: return np.linalg.norm(r), np.inf
    t=-(r@v)/(v@v); t=max(t,0.0)
    cpa=np.linalg.norm(r+v*t)
    return cpa, t

# חמש מטרות: מיקום, מהירות, הסתברות עוינות
tgts=[
```

```
  dict(name='T1', pos=np.array([100.,5.]),  vel=np.array([-10.,0.]),  p_host=0.9),
  dict(name='T2', pos=np.array([80.,80.]),  vel=np.array([-8.,-8.]),  p_host=0.5),
  dict(name='T3', pos=np.array([200.,2.]),  vel=np.array([-5.,0.]),   p_host=0.95),
  dict(name='T4', pos=np.array([30.,60.]),  vel=np.array([0.,-2.]),   p_host=0.2),
  dict(name='T5', pos=np.array([60.,3.]),   vel=np.array([-12.,0.]),  p_host=0.7),
]
for t in tgts:
    cpa,tti=cpa_tti(t['pos'],t['vel'])
    f_cpa=np.exp(-cpa/20.0)           # קרוב יותר -> מסוכן יותר
    f_tti=np.exp(-tti/15.0)           # מוקדם יותר -> דחוף יותר
    t['cpa'],t['tti'],t['threat']=cpa,tti,t['p_host']*f_cpa*f_tti

ranked=sorted(tgts,key=lambda d:-d['threat'])
print('דירוג איום (L3):')
for r,t in enumerate(ranked,1):
    print(f"{r}. {t['name']}  threat={t['threat']:.3f}  CPA={t['cpa']:5.1f}  TTI={t['tti']:
5.1f}  P(host)={t['p_host']}")
```

```
דירוג איום (L3):
1. T5  threat=0.432  CPA=  3.0  TTI=  5.0  P(host)=0.7
2. T1  threat=0.360  CPA=  5.0  TTI= 10.0  P(host)=0.9
3. T2  threat=0.257  CPA=  0.0  TTI= 10.0  P(host)=0.5
4. T3  threat=0.060  CPA=  2.0  TTI= 40.0  P(host)=0.95
5. T4  threat=0.006  CPA= 30.0  TTI= 30.0  P(host)=0.2
```

```
# וידואליזציה של תמונת המצב (L2) ודירוג האיום (L3)
plt.figure(figsize=(7,6))
plt.plot(*asset,'b*',ms=20,label='נכס מוגן')
for t in tgts:
    p,v=t['pos'],t['vel']
    plt.scatter(*p,s=80+400*t['threat'],c=[[t['threat'],0,1-t['threat']]])
    plt.arrow(p[0],p[1],v[0]*4,v[1]*4,head_width=4,color='gray',alpha=.6)
    plt.annotate(f"{t['name']}\n{t['threat']:.2f}",p,textcoords='offset points',xytext=(6,6))
plt.title('תמונת מצב (L2) + הערכת איום (L3)\nאיום גבוה = אדום/גודל');
plt.gca().set_aspect('equal')
plt.xlabel('x'); plt.ylabel('y'); plt.legend(); plt.grid(alpha=.3); plt.tight_layout();
plt.show()
```

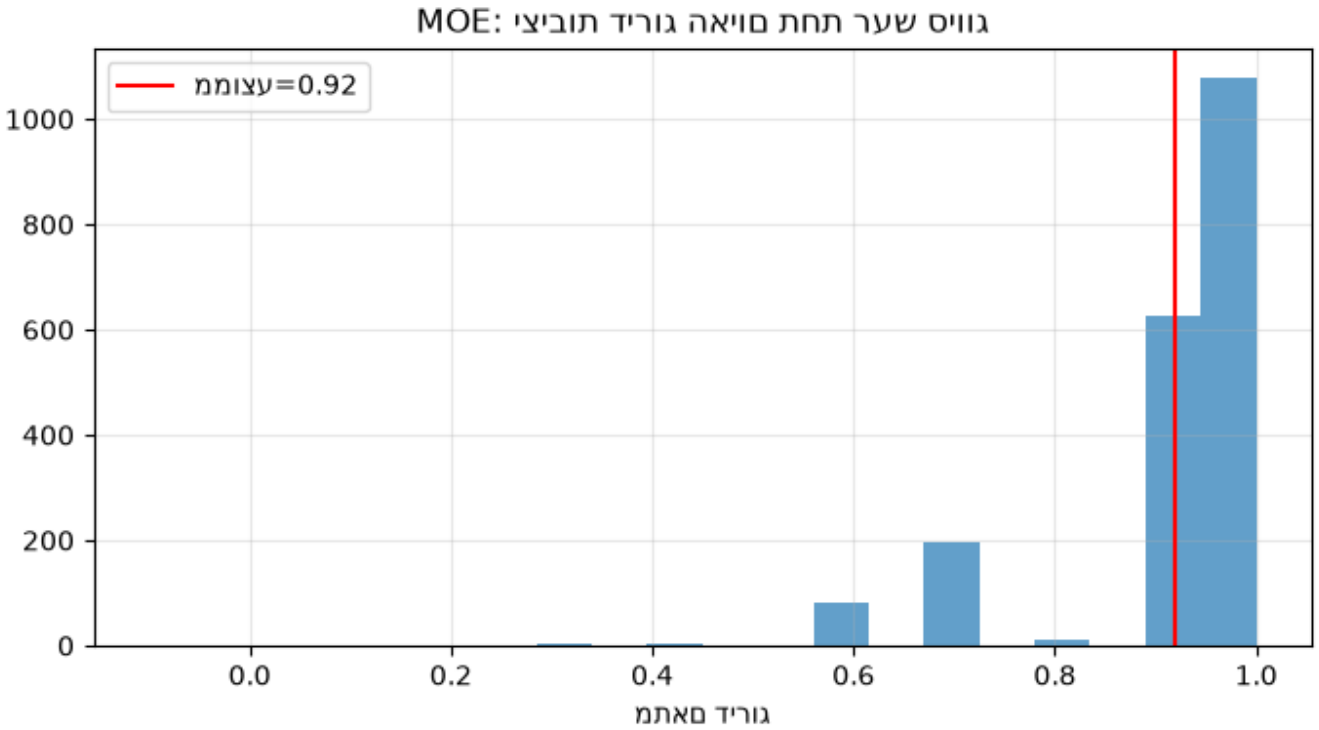


ג. מדדי הערכה (MOP/MOE) — הקושי שהדגיש Blasch¶

Blasch מדגיש ש**הערכת מערכות fusion קשה** (מורכבות, חסינות, יכולת-פעולה-הדדית). מבחינים בין:

- **MOP (Measures of Performance)** — מדדים טכניים נמוכי-רמה: RMSE, P_D/P_{FA}, זמן ריצה.
- **MOE (Measures of Effectiveness)** — האם המשימה הושגה: דיוק דירוג האיום, אזהרה בזמן, עומס על המפעיל. מדד נפוץ לאיכות החלטה: **שטח מתחת ל-ROC (AUC)**.

```
# MOE: האם דירוג האיום שלנו מתאם לאיום ה'אמיתי'? (Monte-Carlo עם רעש סיווג)
def trial():
    order_est=[]; order_true=[]
    for t in tgts:
        p_obs=np.clip(t['p_host']+0.2*rng.standard_normal(),0,1)  # סיווג רועש
        f_cpa=np.exp(-t['cpa']/20); f_tti=np.exp(-t['tti']/15)
        order_est.append(p_obs*f_cpa*f_tti)
        order_true.append(t['threat'])
    # Spearman-like: התאמת סדר העדיפויות
    re=np.argsort(np.argsort(-np.array(order_est)))
    rt=np.argsort(np.argsort(-np.array(order_true)))
    return np.corrcoef(re,rt)[0,1]
corrs=[trial() for _ in range(2000)]
plt.figure(figsize=(7,4)); plt.hist(corrs,bins=20,alpha=.7)
plt.axvline(np.mean(corrs),color='r',label=f'ממוצע={np.mean(corrs):.2f}')
plt.title('MOE: יציבות דירוג האיום תחת רעש סיווג'); plt.xlabel('מתאם דירוג'); plt.legend()
plt.grid(alpha=.3); plt.tight_layout(); plt.show()
```



ד. Soft Fusion + Hard והאדם בלולאה (L5)¶

fusion ברמה גבוהה משלב מקורות **hard** (חיישנים פיזיקליים) עם מקורות **soft** (דיווחי אדם, טקסט, מודיעין). Blasch מדגיש ש-fusion **אינו** מחליף הסקה אנושית אלא תומך בה (L5): הצגה, הסבר, ואינטראקציה. האתגרים הפתוחים שמנה במבוא: מידול **כוונה (intent)**, יריב **תגובתי אינטליגנטי**, וצורך ב**בעיות אתגר משותפות** (common challenge problems) ובהערכה סטנדרטית להשוואה ושחזור.

סיכום¶

- רמות **L2–L5** עוברות מ'עצמים' ל'משמעות': מצב, איום, ניהול תהליך, ואדם בלולאה.
- הערכת איום משלבת **קינמטיקה (CPA/TTI)** עם **זהות (P(hostile))** — חיבור של מחברות 03–07.
- יש להבחין בין **MOP** (ביצועים טכניים) ל-**MOE** (הצלחת המשימה); הערכת fusion ברמה גבוהה נותרת קשה.
- היעד: **hard+soft fusion** התומך בהסקה אנושית — סגירת המעגל חזרה לעקרונות במחברת 01.